Natural Language Processing Journal 12 (2025) 100174

Contents lists available at ScienceDirect

Natural Language Processing Journal

journal homepage: www.elsevier.com/locate/nlp

Distributed training of large language models: A survey

Fanlong Zeng [a], Wensheng Gan [a,*], Yongheng Wang [a], Philip S. Yu [b]

[a] School of Intelligent Systems Science and Engineering, Jinan University, Zhuhai 519070, China
[b] Department of Computer Science, University of Illinois Chicago, Chicago, IL 60607, USA

ARTICLE INFO

Keywords:
Big data
Distributed training
Large language models
Data parallelism
Pipeline parallelism

ABSTRACT

The emergence of large language models (LLMs) such as ChatGPT has opened up groundbreaking possibilities, enabling a wide range of applications in diverse fields, including healthcare, law, and education. A recent research report highlighted that the performance of these models is often closely tied to their parameter scale, raising a pressing question: how can we effectively train LLMs? This concern is at the forefront of many researchers' minds. Currently, several distributed training frameworks, such as Megatron-LM and DeepSpeed, are widely used. In this paper, we provide a comprehensive overview of the current state of LLMs, beginning with an introduction to their development status. We then dig into the common parallel strategies employed in LLM distributed training, followed by an examination of the underlying technologies and frameworks that support these models. Next, we discuss the state-of-the-art optimization techniques used in LLMs. Finally, we summarize some key challenges and limitations of current LLM training methods and outline potential future directions for the development of LLMs.

1. Introduction

In recent years, artificial intelligence (Winston, 1984), including deep learning (LeCun et al., 2015) and data mining (Gan et al., 2023a, 2025a), has been widely applied in various fields. The evolving complexity of models and prolonged training times present considerable challenges to effective model development as technology advances (Li, 2020a; Ananthaswamy, 2023; Brown et al., 2020; Dodge et al., 2022). With the emergence of bidirectional encoder representations from transformers (BERT) (Devlin et al., 2018) and ChatGPT (Brown et al., 2020), large language models (LLMs) (Wu et al., 2023; Gan et al., 2023c) have reached parameter sizes in the trillions, marking the era of large models. In recent years, LLMs have received great attention and rapid development. It has been initially used in various fields, including education (Gan et al., 2023b; Xu et al., 2024), medicine (Zheng et al., 2025), recommender systems (Zhu et al., 2024b), robotics (Zeng et al., 2023a), law (Lai et al., 2024), chemical science (Boiko et al., 2023), etc. Nowadays, pre-trained models often suffer from a lack of accuracy in vertical domains, which can limit their effectiveness in specific applications. As a result, some research on LLMs is shifting its focus towards post-training domain adaptation techniques. However, distributed training (Zeng et al., 2023b) remains an evolving field, particularly in the pre-training phase of LLMs. In this era, the expansion of the model presents new requirements for hardware development. The capacity and computing power of a single device are insufficient for training LLMs. Therefore, some studies have turned to multi-node distributed training in clusters, which has become a necessity for training large models.

Despite the explosive growth in model size, hardware development has been somewhat insufficient and stagnant. Training LLMs requires more than just increasing the number of devices; it necessitates specific parallel and communication strategies. In the field of LLM, distributed training (Li et al., 2014; Gibiansky, 2017; Luo et al., 2020; Rajbhandari et al., 2020) is an important technical method for training huge models. It involves distributing the training work to multiple computing nodes, using effective communication and coordination mechanisms to enable them to work together. Through distributed training, the computing resources in the cluster can be fully utilized. Distributed training can enhance training efficiency and process larger datasets more effectively. Therefore, it has attracted a lot of research on relatively efficient model training with limited resources. Based on distributed training, various parallel strategies have been proposed, including data parallelism[1] (Li et al., 2014; Gibiansky, 2017; Luo et al., 2020; Rajbhandari et al., 2020), model parallelism (Huang et al., 2019; Narayanan et al., 2019, 2021), 3D parallelism (Lai et al., 2023; Rasley et al., 2020), etc. In addition, various training optimization strategies have also been proposed, such as model compression (Ma et al., 2023; Frantar and Alistarh, 2023; Zhang et al., 2023), reduced activation calculations (Chen et al., 2016; Korthikanti et al., 2023), mixed precision calculations (Narang et al.,

* Corresponding author.
E-mail addresses: flzeng1@stu.jnu.edu.cn (F. Zeng), wsgan@jnu.edu.cn (W. Gan), yonghengwang@stu2023.jnu.edu.cn (Y. Wang), psyu@uic.edu (P.S. Yu).

[1] Notice that the terms, data parallel, and data parallelism, are used interchangeably in this paper.

https://doi.org/10.1016/j.nlp.2025.100174
Received 8 November 2024; Received in revised form 12 June 2025; Accepted 13 July 2025

2949-7191/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

2018), optimizer strategies (Schaul et al., 2013; Khirirat et al., 2017; Qian, 1999; Duchi et al., 2011), etc. By adopting these strategies, LLMs can be trained relatively efficiently.

Although distributed training and optimization strategies can improve the training efficiency of large-scale language models in practice, they still have some challenges. In the era of LLM, we have a large number of datasets that require pre-training for LLM, and this is vital to its performance. To obtain a stronger performance of LLM in specific downstream tasks, we may need to fine-tune it. Should we fine-tune it, and how many datasets should we use to optimize performance in designated downstream tasks? Will our privacy be leaking? We also lack a resource-friendly way to train LLM; it is too costly to train it just once. Nowadays, with the appearance of ChatGPT, people have started using the API of LLM. It may be a tendency for humans to utilize LLM, but sometimes it has a large latency when we use it. LLM also has discrimination problems, when it learns from various datasets that include bias. LLM gives us a very surprising performance, but the content it creates may have issues, like hallucination or harmful content. Moreover, it is crucial to ensure the safety of LLMs by aligning them with human values and maintaining control over their development and deployment. We get more than these challenges that need to be solved in the future.

This review aims to provide a comprehensive yet concise overview of the LLM field, with a specific focus on distributed training, thereby catering to the needs of researchers interested in this domain. The survey delves into the underlying technologies that facilitate distributed training, including frameworks, strategies, and optimization techniques, while also examining the associated challenges. By presenting a detailed analysis of the current landscape and future directions, this paper seeks to inform and guide researchers in their exploration of LLM-distributed training. The contributions of this review are as follows:

- This paper begins by defining LLM, then proceeds to categorize and describe various optimization strategies for distributed LLM (Section 2). We provide a comprehensive comparison between the popular distributed inference and training framework (Section 2.2 to Section 2.4).
- We offer a thorough review of each training strategy, providing a detailed background, key components, problem-solving capabilities, and concise explanations of the relevant content (Section 2.5). We also compare the differences among the techniques (Section 2.6).
- Besides discussing the distributed optimization strategy of LLM, we also explore the underlying architecture and fundamental technologies that LLM adopts, intending to provide readers with a comprehensive understanding of LLM's evolution and current state (Section 2.7).
- We also summarize the challenges encountered in the current LLM training process and the possible improvement directions for future distributed training of LLM (Section 3). These should help us gain a comprehensive understanding of the challenges encountered in the current LLM training process (Section 4).

Note that this paper is a comprehensive survey of distributed training of LLMs, as an extension of the brief conference version (Zeng et al., 2023b). Section 2 provides an in-depth examination of distributed LLMs, including the techniques used to accelerate training, such as parallel strategies, architectural designs, underlying technologies, and optimization methods. We compare different frameworks and introduce the newest frameworks emerging in recent years. Section 3 discusses the current challenges and limitations of LLM training. Finally, Section 4 summarizes this article and outlines several potential future directions for the development of LLMs.

2. Preliminaries

2.1. Survey methodology

This section outlines the methodology used to collect and select the relevant articles included in this survey, providing a transparent and systematic approach to our review of the literature. Our literature search primarily utilized Google Scholar,[2] and DBLP[3] with 'distributed training LLM' as the primary keyword, and focused on relevant articles published between 2020 and 2025. Notably, we also included a selection of seminal articles from earlier years. When selecting citations, our primary criterion is the article's impact, as measured by its citation count. For the cited article, we primarily employed a manual classification approach, categorizing papers based on their relevance to the definition, framework, strategies, and optimization methods of distributed training of LLMs. Additionally, we provided an overview of the basic architecture of LLMs, laying the foundation for a comprehensive understanding of the field.

2.2. Distributed parallel inference framework

vLLM (Kwon et al., 2023) is a cutting-edge LLM inference engine developed by the University of California, Berkeley. This high-performance engine is designed to efficiently deploy and serve LLMs, making it an ideal solution for applications that require supporting multiple user requests and dynamic batch processing. To achieve optimal performance, vLLM employs various innovative techniques, utilizing PagedAttention to manage attention mechanisms and 1D tensor parallelism, which significantly improves inference throughput. While vLLM's memory management advantages are most pronounced for LLMs, the system's benefits may be less significant for smaller models.

Petals (Borzunov et al., 2023) is an open-source, collaborative LLM distributed inference system developed by the BigScience community. Its primary objective is to democratize access to LLM by enabling anyone to co-host and operate a model, regardless of their computational resources. To achieve this, Petals employs a novel approach called Collaborative Inference, which shards a large model into smaller components and distributes them across a network of volunteer-hosted servers or individual GPUs. The inference results are then aggregated through a decentralized network, eliminating the need for costly centralized server clusters and reducing the computational burden. This design allows anyone with a GPU to join the network and contribute to the model's operation, making it a highly inclusive and scalable solution. However, the system's distributed nature introduces latency due to cross-network communication between layers, resulting in inference speeds that are significantly slower than those achieved through local deployment.

HuggingGPT (Shen et al., 2023) is a pioneering system proposed by the Microsoft team, which integrates LLM with a multitude of AI tool models on Hugging Face to create a universal AI assistant system capable of executing cross-modal tasks. At its core, HuggingGPT is a task scheduling system that leverages LLM to orchestrate model calls across different servers during inference, thereby enabling distributed task execution. However, the current implementation involves sequential model calls, with each step requiring remote communication, resulting in prolonged response times and relatively slow inference times due to the overhead of repeated network requests.

2.3. How to increase the training speed of LLM?

To enhance the training speed of LLM, distributed training is a

[2] https://scholar.google.com/
[3] https://dblp.org/

crucial approach. The primary objective of distributed training is to boost overall training speed, raising the key question: how can we optimally improve the model's training speed? Here we have a formula for the total training speed of the model: Total Training Speed = Single GPU Speed × Number of Accelerator Chips × Speed-Up Ratio of Multiple GPUs.

Single-GPU speed is primarily determined by data IO speed and computing speed. It is essential to optimize the training process on a single GPU. Theoretically, increasing the number of accelerator chips for training speeds up the training process. However, as the number of chips surpasses a certain threshold, it leads to substantial communication overhead, which becomes a bottleneck. Further increasing the number of chips (Number Of Accelerator Chips) do not accelerate training. This diminishing effect on computing resources is addressed by optimizing server architecture and communication topology to improve communication efficiency. The speed-up ratio of multiple GPUs is jointly determined by computing and communication efficiency, requiring optimization alongside the algorithm and cluster network topology. Strategies like data and model parallelism enhance multi-GPU training efficiency. The primary objective of LLM distributed training is to optimize the formula above, effectively splitting data and models across diverse devices with limited resources, while enhancing communication efficiency between clusters. The following sections introduce current parallel strategies, the fundamental techniques, and the architectures used in LLM. Furthermore, we provide an overview of the most widely used framework for distributed data parallelism (DDP) in Table 1.

2.4. Distributed parallel training framework

2.4.1. Main section

DeepSpeed (Rasley et al., 2020) provides a GPT-like model training method with a single click, offering incredible acceleration and unprecedented overhead reduction. It is specifically designed for training LLMs. Compared with other frameworks, DeepSpeed supports larger-scale models and provides more optimization strategies and tools. Using 3D parallelism to achieve trillion-parameter model training, ZeRO-Offoad (Ren et al., 2021) enables a single GPU card to train 10x models. With the sparse attention mechanism supporting the input of long sequence models, DeepSpeed also reduces a large amount of communication compared to ADAM (Kingma and Ba, 2014). DeepSpeed features a ZeRO (Rajbhandari et al., 2020) series optimizer that significantly reduces memory usage during training. Additionally, it exhibits excellent scalability, supporting the combination of four parallelization methods: data parallelism, model parallelism, pipeline parallelism, and tensor parallelism. Moreover, DeepSpeed allows for substantial training acceleration through the utilization of mixed precision, communication compression, optimizer sharding, and other optimization techniques. However, configuring DeepSpeed requires a multitude of complex hyperparameters, which can be challenging to adjust. Furthermore, DeepSpeed's performance on heterogeneous hardware may be compromised, as its ZeRO optimizer offloads certain computations to the CPU and NVMe storage. If these components are of lower quality or insufficient capacity, training efficiency can be negatively affected.

Megatron-LM (Narayanan et al., 2021) is a strong transformer-based framework developed by the NVIDIA team. Megatron-LM is known for its impressive scale, capable of handling models with billions of parameters. It leverages distributed training techniques to efficiently train such LLMs across multiple GPUs or even multiple machines. This allows researchers and developers to tackle complex NLP tasks that require massive computational resources. Megatron-LM provides a powerful framework for building and training large-scale language models, pushing the boundaries of what is possible in natural language processing and enabling breakthroughs in various NLP applications. Megatron-LM exhibits exceptional scalability, enabling the training of massive models with over 100 billion parameters on thousands of GPUs. Additionally, Megatron-LM supports 3D parallelism, which combines data parallelism, tensor parallelism, and pipeline parallelism at the same time. Moreover, with the help of 3D parallelism, Megatron-LM can maximize GPU utilization and optimize training efficiency. However, configuring Megatron-LM is a complex task that requires a deep understanding of distributed training, CUDA, and communication strategies. Furthermore, compared to more comprehensive ecosystems like Hugging Face, Megatron-LM's task coverage is relatively limited, making it more suitable for large-scale pre-training scenarios rather than downstream fine-tuning tasks.

Colossal-AI (Li et al., 2023) provides a collection of parallel components to support users in writing distributed deep learning models as easily as they write models on their laptops. Compared with existing solutions, the parallel dimensions only include data parallelism, one-dimensional tensor parallelism, and pipeline parallelism. Colossal-AI further provides 2-, 2.5-, and 3-dimensional tensor parallelism and sequence parallelism, as well as convenient multi-dimensional hybrid parallelism solutions. It integrates multiple memory optimization technologies, including multi-dimensional parallelism, ZeRO redundant memory elimination (Rajbhandari et al., 2020), CPU offload, Gradient Checkpoint (Chen et al., 2016; Korthikanti et al., 2023), automatic mixed precision (AMP) (Kotipalli et al., 2019), and other cutting-edge technologies to help users avoid memory bottlenecks. It reduces training hardware requirements to the greatest extent. The open-source community of Colossal-AI is vibrant, well-documented. Colossal-AI also leverages ZeRO to substantially reduce training memory requirements, making it an attractive option for LLM training. Additionally, Colossal-AI exhibits high scalability, supporting multiple parallel strategy combinations, and can seamlessly adapt to various cluster sizes and resource configurations. The parameter configuration is relatively straightforward and user-friendly, particularly when compared to Megatron-LM. Moreover, Colossal-AI also supports automatic parallel splitting and scheduling. However, some optimizations are tailored to specific hardware or CUDA features, which may limit their effectiveness on non-NVIDIA platforms. While Colossal-AI does support Hugging Face, it may require additional fine-tuning and adaptation to accommodate different model fine-tuning paths. Furthermore, its overall maturity is slightly lower than DeepSpeed and FSDP.

Megatron-DeepSpeed (NVIDIA, 2023) incorporates supplementary support for several features. It combines the power of Megatron-LM with the optimization capabilities of DeepSpeed, a library specifically designed to improve training efficiency and scalability for deep learning models. Megatron-DeepSpeed offers a powerful framework for training large-scale language models efficiently. It empowers researchers and practitioners to advance the frontiers of language comprehension and production, allowing them to explore new possibilities and applications in the field of NLP. It also paves the way for groundbreaking advancements in natural language processing tasks. Megatron-DeepSpeed exhibits exceptional scalability, supporting a combination of tensor parallelism, pipeline parallelism, and ZeRO, making it an ideal choice for leveraging diverse cluster resources. The communication efficiency of Megatron-DeepSpeed is particularly noteworthy, with optimized support for the NCCL communication topology and DeepSpeed, which significantly reduces synchronization costs. However, configuring Megatron-DeepSpeed can be complex, requiring careful combination and debugging of tensor, and pipeline parallelism. Furthermore, it is heavily reliant on CUDA and NCCL, with limited support for AMD or non-CUDA backends, which may restrict its deployment on certain hardware configurations. Additionally, Megatron-DeepSpeed stringent hardware communication bandwidth requirements, necessitating high-speed interconnects between GPUs, such as NVLink or Infiniband, to maintain optimal efficiency.

LightSeq2 (Wang et al., 2021) is a high-performance, open-source training library developed by Bytedance, specifically designed to accelerate Transformer-related models. It supports both tensor parallelism and pipeline parallelism, making it well-suited for distributed training scenarios of LLMs. Lightseq2 leverages a fused CUDA kernel

Table 1
The framework of DDP LLM training.

Company	Framework	Description
Microsoft	DeepSpeed (Rasley et al., 2020)	DeepSpeed provides ChatGPT-like model training with just one click, offer--ing unimaginable acceleration and unprecedented overhead reduction.
NVIDIA	Megatron-LM (Narayanan et al., 2021)	Megatron-LM is a strong transformer-based model developed by NVIDIA.
HPC-AI Technology	ColossalAI (Li et al., 2023)	Colossal-AI provides a collection of parallel components for users, aiming to support the user to write their distributed deep learning models just like how they write their models on laptops.
Microsoft	Megatron-DeepSpeed (NVIDIA, 2023)	Megatron-LM's DeepSpeed version that incorporates supplementary support for several features, such as the MoE model training, curriculum learning, 3D parallelism, etc.
HuaWei	MindSpeed (MindSpeed-LLM, 2023)	MindSpeed is an open-source framework that delivers high-performance capabilities for accelerating model training and inference, developed by Hua--wei Ascend team.

Table 2
The summary of DDP LLM training.

Year	Method	Description
2014	Parameter Server (Li et al., 2014)	PS architecture is P2P communication; PS nodes will be divided into server nodes and work--er nodes. The server node will maintain the parameters of the model, and each worker will have a part of the training data to perform corresponding calculations. Workers only comm--unicate with server nodes.
2017	Ring All-Reduce (Gibiansky, 2017)	Ring All-Reduce technique to reduce the communication overhead between GPUs in the cl--uster and allow more time to be used in effective computing.
2020	Partial All-Reduce (Luo et al., 2020)	Partial All-Reduce is a new communication primitive, which can achieve rapid synchronization of a group of workers, and is a decentralized method.
2020	ZeRO (Rajbhandari et al., 2020)	ZeRO can optimize memory and significantly increase the training speed on the premise of increasing the model size for efficient model training.
2021	ZeRO-Offload (Ren et al., 2021)	ZeRO-Offload can greatly reduce the communication overhead between CPU and GPU. ZeRO-Offload offloads some calculations to the GPU for LLM training and can use CPU memory and calculations for offloading at the same time.
2021	ZeRO-Infinity (Rajbhandari et al., 2021)	ZeRO-Infinity is a brand new hybrid system that weights GPU, CPU, and NVMe memory are used to train unprecedented models with limited resources.
2023	FSDP (Zhao et al., 2023)	FSDP enables the training of LLMs by partitioning model parameters, utilizing a communi--cation-based strategy to coordinate the parameters.

and a custom memory allocator to achieve significant performance gains over PyTorch's default implementation. Additionally, it supports multiple parallelization modes, including DDP-like tensor or pipeline parallelism. However, fully harnessing its potential requires a solid understanding of CUDA kernels and distributed training mechanisms. Furthermore, the Lightseq2 ecosystem is relatively niche, lacking a comprehensive memory management and scheduling mechanism comparable to those found in more established libraries like DeepSpeed and FSDP. As a result, users may need to invest additional time and effort into optimizing and fine-tuning their workflows.

MindSpeed (MindSpeed-LLM, 2023) is an open-source, high-performance framework for accelerating model training and inference, developed by the Huawei Ascend team. Designed to provide comprehensive support for end-to-end model, training, and deployment on Ascend AI chips. MindSpeed offers a robust ecosystem for LLM acceleration. By leveraging automatic parallelism, memory optimization, and mixed precision techniques, MindSpeed significantly enhances the trainability and performance of LLMs on ascend hardware in distributed training environments. MindSpeed is highly optimized for Huawei Ascend processors and demonstrates exceptional efficiency on domestically produced hardware in China. By integrating data parallelism, model parallelism, pipeline parallelism, and ZeRO, MindSpeed enables the training of large-scale parameter models. However, MindSpeed's compatibility with PyTorch is limited, as the framework is heavily inclined towards MindSpore, which is not part of the native PyTorch ecosystem. This results in significant learning and transfer costs for PyTorch users, who may need to invest considerable time and effort to adapt to the MindSpore ecosystem. Furthermore, MindSpeed's optimal performance is closely tied to the ascend series of devices, which may limit its cross-platform compatibility and hinder deployment on non-Ascend hardware.

2.4.2. Simplified training framework

Due to the complexity of parameter configuration and training processes in deep learning frameworks, several libraries have emerged to simplify training logic and accelerate the development process. In this section, we introduce Ray (Moritz et al., 2018) and Hugging Face Accelerate (Gugger et al., 2022), which are designed to streamline the training workflow and improve overall efficiency. Moreover, Table 3 lists the main distributed training frameworks of LLMs, comparing their characteristics.

Ray (Moritz et al., 2018) is an open-source, general-purpose distributed computing framework initially developed by UC Berkeley's RISELab and subsequently commercialized by Anyscale. With a design goal of simplifying distributed computing to the ease of writing Python, Ray offers efficient and scalable support for training, fine-tuning, and deploying LLMs through its suite of components, including Ray Train, Ray Data, and Ray Serve. While Ray does not directly participate in tensor or pipeline parallelism, it is positioned as a foundational framework, serving as the underlying coordinator for distributed training systems. Nevertheless, the reliance of Ray on actor and object store mechanisms can lead to substantial memory consumption, particularly in scenarios involving high task intensity or multi-node deployments. Additionally, during large-scale task scheduling, the latency of scheduling may emerge as a significant performance bottleneck.

Hugging Face Accelerate (Gugger et al., 2022) is a lightweight library developed by the Hugging Face team, designed to streamline

the complexity of multi-device (CPU, GPU, and TPU) and distributed training. By encapsulating popular PyTorch strategies such as DDP, FSDP, TPU, and DeepSpeed, accelerate eliminates the need for users to write intricate initialization and synchronization logic. Furthermore, it simplifies distributed configuration through a user-friendly interface, allowing users to easily set up DDP, FSDP, and other settings via a configuration file. However, Hugging Face Accelerate has limitations in supporting complex parallelization methods, such as model parallelism and tensor parallelism. Additionally, it is not well-suited for parallelizing the optimization of LLMs, which often require sophisticated parallelization techniques to mitigate communication and memory bottlenecks. Specifically, it may struggle to fully optimize the communication and memory usage of models with billions of parameters.

2.4.3. Emerging frameworks

In this section, we introduce the up-to-date distributed training framework. We divide the current distributed training framework into parallel training strategies and system optimization, asynchronous and elastic training methods, federated learning and decentralized training, and training efficiency and resource optimization.

Parallel training strategies and system optimization. These techniques enable the efficient execution of training processes on multiple devices, such as GPUs or computing nodes, by splitting and parallelizing the training tasks of LLMs (Wagenländer et al., 2024; Gumaan, 2025; Sun et al., 2024; Zhu et al., 2025). These strategies include data parallelism, model parallelism, pipeline parallelism, etc. Several new technologies are currently emerging. CO2 (Sun et al., 2024), for example, introduces the concept of full overlap—a novel approach that decouples communication and computation operations across the four stages of deep learning training: forward propagation, backward propagation, all-reduce, and optimizer step. To enable this overlap, CO2 employs a dual buffer pipeline, in which computing tasks are divided into micro-batches, and the computation of each micro-batch is executed in parallel with communication between adjacent micro-batches. Additionally, CO2 utilizes non-blocking communication mechanisms, such as CUDA Streams and NCCL communication library, to ensure that communication operations do not hinder the computing stream. However, CO2 may introduce implicit race conditions in completely asynchronous execution. Mist (Zhu et al., 2025) introduces a novel approach called memory parallel learning co-optimization, which dynamically allocates parameter storage locations based on the unique characteristics of each model layer, including the number of attention heads and FFN layer width, and NVMe storage. This approach enables the automatic selection of the optimal parallelization strategy for each layer, allowing for a hybrid parallelization approach that combines different techniques. However, it is worth noting that Mist is specifically optimized for Transformer-class models and may not be directly applicable to other architectures.

Asynchronous and elastic training methods. They are two complementary approaches designed to optimize the efficiency of training workloads (Huang et al., 2024; Cheng et al., 2024; Wu et al., 2025). Asynchronous training aims to eliminate synchronous waiting, such as AllReduce blocking (Gibiansky, 2017). Elastic training, on the other hand, focuses on ensuring uninterrupted training, enabling automatic recovery in the event of node failures, and facilitating the dynamic expansion of GPU resources. Despite their distinct objectives, both approaches share a common requirement: ensuring training consistency. EDiT (Cheng et al., 2024) introduces an efficient distributed training framework built based on Local SGD, proposing Hierarchical Gradient Aggregation to facilitate local gradient updates within workers and synchronous gradient synchronization across workers. Furthermore, EDiT proposes an adaptive strategy that dynamically adjusts the synchronization interval based on gradient variance, which is achieved by monitoring the deviation between local gradients and global parameters. This approach enables the framework to strike a balance between local updates and global synchronization, optimizing the trade-off between convergence speed and communication overhead. However, it is important to note that excessive local update steps can potentially compromise convergence. The Ant Group has developed a secure distributed framework (Huang et al., 2024) for LLM training, designed to overcome the limitations of traditional distributed training methods. This framework employs a multi-faceted approach, combining model partitioning, trusted execution environments (TEE) (Huang et al., 2024), and lightweight encryption to ensure the secure and efficient training of LLMs with billions of parameters. A key feature of this framework is its support for adaptive hybrid parallelism, which enables the dynamic switching between data, model, and pipeline parallel combinations to optimize training performance. Moreover, the framework incorporates a secure training protocol to protect sensitive data, providing a robust and reliable solution for LLM training.

Federated learning and decentralized training. Federated learning (FL) (Chen et al., 2022, 2023) enables privacy-preserving model training by retaining data on local devices and only uploading model gradients or parameters to a central server for aggregation (Iacob et al., 2024; Ye et al., 2024; Sani et al., 2024). FL can be broadly categorized into two architectural frameworks: centralized and decentralized. In the centralized architecture, a central server coordinates global model updates by aggregating and broadcasting model parameters to participating nodes. In contrast, the decentralized architecture enables nodes to communicate directly with each other. OpenfedLLM (Ye et al., 2024) introduces a framework for federated training of LLMs with billions of parameters. This innovative architecture proposes a hierarchical federated training approach, where clients utilize LoRA to update only low-rank matrices, reducing the computational burden and communication overhead. Geographically adjacent devices are locally aggregated through P2P networks. The server then integrates the various cluster models through knowledge distillation, generating a global LLM that leverages the collective knowledge of the participating devices. To further optimize the communication process, OpenFedLLM transmits only the top 1% of significant gradients, significantly reducing the communication overhead. Moreover, the framework implements asynchronous updates, allowing lagging devices to participate in the training process. OpenFedLLM strengthens privacy protection. Recent research (Sani et al., 2024) proposes that the training of LLMs in the future may adopt a federated approach, as current training methods include risks of data monopolization and non-compliance with privacy regulations.

Training efficiency and resource optimization involve the development of innovative algorithms, systems, and hardware architectures that enhance the training process of LLM. The primary goal of this effort is to significantly improve training speed, reduce resource consumption, and maximize resource utilization (Song et al., 2023; Li et al., 2024; Chen et al., 2025). To achieve this, various techniques are employed, including mixed precision and compute-comm overlap. Conventional LLM training protocols typically require full parameter updates, which incur substantial computational overhead due to the sheer volume of parameters involved. To address this issue, some new training paradigms have been proposed in recent years. EST (Li et al., 2024) introduces a novel dynamic subnet evolution framework, featuring a two-stage loop that enables efficient and adaptive training of deep neural networks. The exploration phase involves randomly activating different subnets, consisting of a subset of parameters (e.g., 5%), every K steps, and evaluating their importance through gradient signals. In the utilization stage, the most important parameters (e.g., 1%) are locked, and the remaining parameters are dynamically rotated for training. Furthermore, EST has undergone system optimizations, including retaining only the optimizer state of active parameters, which reduces memory usage by 70%, and synchronizing only the gradients of active subnets in distributed training, resulting in a 10-fold reduction in communication overhead.

Table 3
Comparison of distributed training frameworks.

Framework	Scalability	Efficiency	Typical use cases
DeepSpeed	Leverages ZeRO, ZeRO-Offload, ZeRO-Infinity, and pipeline parallelism for efficient multi-GPU distributed training.	Optimizes memory with mixed precision training, communication overlap, and compression; ZeRO memory optimization.	Training ultra-large LLMs, inference optimization, and fine-tuning.
Megatron-LM	Utilizes Tensor and pipeline parallelism; supports 3D parallelism.	Improves memory usage via Fused Ops and overlapping communication/computation.	Pretraining large autoregressive transformer models (e.g., GPT, BERT-like).
Colossal-AI	Supports 1D/2D/2.5D/3D parallelism; compatible with ZeRO; integrates Megatron-LM and DeepSpeed.	Employs mixed precision training and dynamic memory management.	Cost-efficient training on heterogeneous hardware.
Megatron-DeepSpeed	Supports 3D parallelism; integrates ZeRO stages 1–3 and ZeRO-Offload.	Uses activation checkpointing and mixed precision training.	Training massive models (e.g., 530B) and LLM serving.
MindSpeed	Designed for Ascend AI processors; supports 3D parallelism and adapts to ascend computing clusters.	Minimizes communication via Ascend's HCCL; uses pipeline reconciliation and activation offload.	LLM training/inference on Huawei Ascend; integrates with MindSpore.

2.5. Distributed parallel strategy

This section introduces the strategies adopted in LLMs, including data parallelism, model parallelism, and 3D parallelism. Fig. 1 classifies distributed parallel strategies discussed in this paper, while Table 2 summarizes the key aspects of DDP training.

2.5.1. Data parallel

Single-machine data parallelism. In deep learning, there is a growing demand for large datasets and models, as well as increased computing resources to support their training. Due to the simple and intuitive principle, data parallelism is widely used in distributed training. Data parallelism is a single-process multi-threaded method that uses multiple GPUs for training on a single machine. The input and model are distributed from the main GPU to all GPUs. Each GPU performs forward propagation independently to obtain results. The distributed GPUs transmit the computed results back to the main GPU and then utilize these outputs to compute the loss, which is subsequently distributed to all GPUs. All GPUs calculate the gradient through backpropagation, and these gradients are sent back to the main GPU for model weight updates. This process is repeated until convergence. For instance, as illustrated in Fig. 2, the input tensor is partitioned into two segments, which are then distributed across different GPUs for parallel computation. The partial results are subsequently combined after being processed on their respective devices. However, data parallelism (DP) has certain disadvantages. Firstly, DP is a single-machine multi-threading mechanism, and it is only suitable for single-machine situations. Secondly, the amount of data transmitted by DP is too large, which results in unbalanced overhead. Third, it does not support model parallelism. It has been abandoned at present.

Distributed data parallelism (DDP) has been proposed to solve the above shortcomings of DP. At present, there are three main methods to implement DDP, namely parameter server (PS), ring All-Reduce, and partial All-Reduce, which are introduced below.

(1) Parameter server (PS) (Li et al., 2014). Distributed optimization is the premise for solving the problem of large model training. As data and model complexity increase, the need for a larger number of parameters arises, and training datasets can easily reach sizes of 1 TB or even 1 PB. Large model training usually faces many challenges. Accessing data requires enormous network bandwidth, and many machine-learning algorithms are also sequential. The challenges mentioned above cost too much for synchronization and latency, which greatly impair the performance of training. Many algorithms are also without fault tolerance (especially in real-world deployments). As illustrated in Fig. 3, the parameter server (PS) architecture employs a peer-to-peer (P2P) communication paradigm, where PS nodes are divided into two categories: server nodes and worker nodes. The server nodes are responsible for maintaining the model's parameters, while each worker node is assigned a subset of the training data to perform computations. Communication occurs exclusively between worker nodes and server nodes. To update the model parameters, nodes utilize push and pull operations. Specifically, worker nodes push the gradients computed during forward propagation to the server nodes, which then aggregate the gradients during back-propagation to update the parameters. Subsequently, worker nodes pull the updated parameters from the server node to continue training. Nevertheless, the PS architecture suffers from high network communication overhead, as workers frequently upload and download gradient parameters, leading to potential network congestion. Furthermore, servers can become bottlenecks, as certain parameters are updated at high frequency, resulting in an uneven load distribution and negatively impacting overall system performance. The communication between workers and servers is characterized by high frequency, with a substantial number of parameter updates and synchronizations occurring in a short period. Additionally, as the number of GPUs or workers increases, the parameter servers do not scale linearly, making it challenging to handle LLMs. This is particularly problematic for LLMs, which possess an enormous number of parameters, rendering a single server incapable of accommodating or managing the vast parameter tensors.

(2) Ring All-Reduce (Gibiansky, 2017). To achieve high -performance computing in the field of deep learning, Ring All-Reduce is utilized as a technique to reduce the communication overhead between GPUs in the cluster and allow more time to be used in effective computing. In the original distributed stochastic gradient descent (SGD), all GPUs have the same model, where the data will be divided into mini-batches and distributed to each GPU. Each GPU will utilize a mini-batch to train independently, performing forward propagation to compute gradients, which are then used to update parameters through backpropagation. This has a significant disadvantage: one must wait until all GPUs finish back-propagation before updating parameters, similar to using only one GPU. When dealing with gigabytes of training data, coordinating communication among multiple GPUs becomes crucial, and the communication mechanism between GPUs becomes particularly important. The cost of communication will increase linearly with the number of GPUs. A simple way is to use an asynchronous method, but this will cause model debugging difficulties, model coverage difficulties, etc. Ring All-Reduce adopts synchronized communication, which is an algorithm whose communication cost in the system is constant and independent of the number of GPUs. It only depends on the slowest link of GPUs in the system. If bandwidth

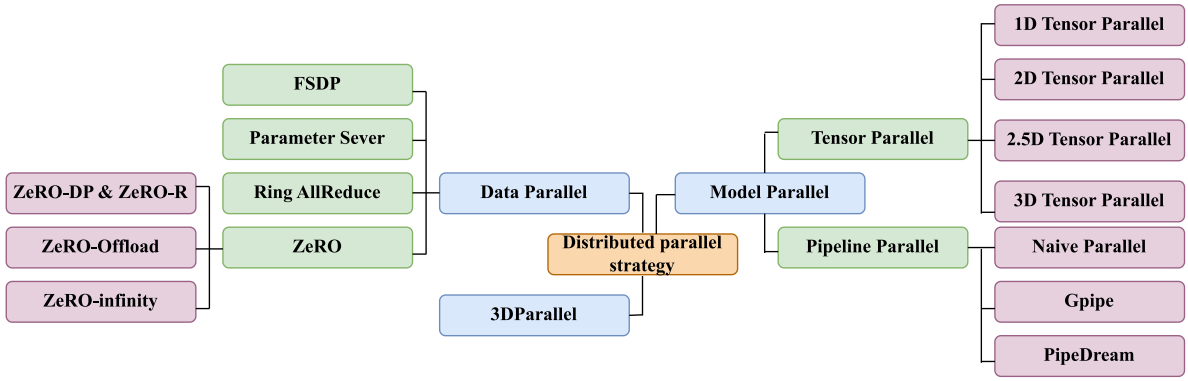


Fig. 1. Classification of distributed parallel strategies discussed in our paper.

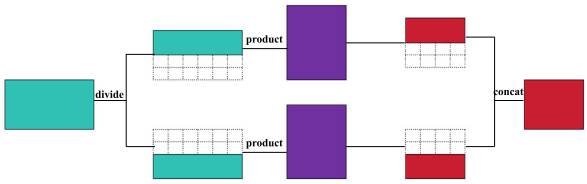


Fig. 2. Data parallelism.

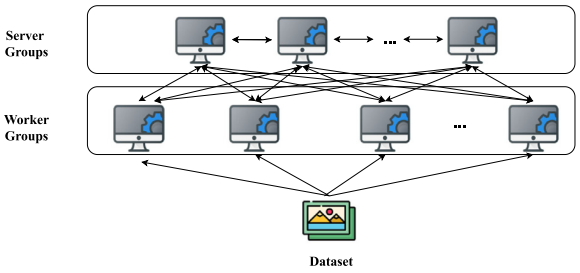


Fig. 3. Parameter server structure.

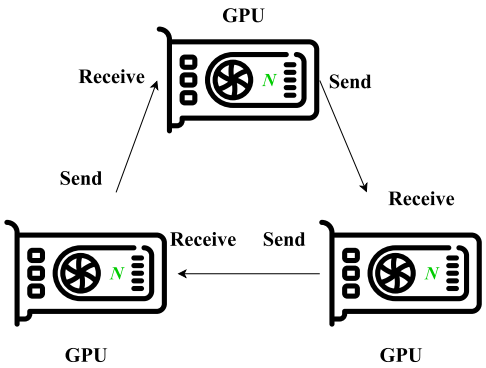


Fig. 4. Ring All-Reduce logical ring.

is the only factor considered as communication overhead, Ring All-Reduce is the idealized algorithm. The GPU in Ring All-Reduce is a logical ring, like in Fig. 4. Each GPU is accompanied by a neighboring GPU to its left and right. A GPU will only send messages to the left neighbor and receive messages from the right neighbor. The algorithm includes two steps, *Scatter-Reduce* and *All-gather*. Under *Scatter-Reduce*, each GPU will exchange data with its right neighbor, and finally, each GPU will have a large piece of data. To simplify, assume that the operation is simply to "add up", where each GPU obtains data from its neighboring GPU on the left and transmits data to its neighboring GPU on the right in each cycle. Each cycle sends out the data block it received in the previous cycle. Under *All-gather*, GPUs will continue to exchange block data from the previous step, and finally, all GPUs will have a complete dataset. After scatter-reduce is completed, there will be a series of values. The process of *All-gather* is the same as scatter-reduce (each iteration will receive a message from the left neighbor, and then send a message to the right neighbor). The only difference is that the current processing method has changed, and the data sent is no longer "addition", but coverage. However, Ring All-Reduce presents significant challenges in terms of fault tolerance and dynamic scheduling. Specifically, if a node fails, the entire loop communication is interrupted, disrupting the training process. Moreover, Ring All-Reduce is not well-suited for heterogeneous devices, as slower nodes can significantly degrade the overall communication efficiency, thereby hindering system scalability. Additionally, the algorithm requires each node to communicate frequently with its two neighbors, demanding high bandwidth and imposes strict topology requirements. This can result in significant communication overhead, especially in large-scale distributed training environments.

(3) Partial All-Reduce (Luo et al., 2020). In the same environment, using *All-Reduce* as the synchronization mechanism for data parallelism results in higher performance. However, the performance of the entire algorithm may be affected by the slowest worker due to hardware and network bandwidth differences. AD-PSGD (Lian et al., 2018) has good heterogeneity tolerance but has deadlock problems and high communication overhead. Is it possible to have a distributed training method that has the efficient performance of All-Reduce and the heterogeneous tolerance of AD-PSGD? Therefore, a new communication primitive Partial All-Reduce, namely P-Reduce was proposed, which can achieve rapid synchronization of a group of workers and is a decentralized method (Luo et al., 2020). In AD-PSGD, if there is a synchronization conflict, that is, $worker_0$ and $worker_1$ select $worker_2$ at the same time for parameter averaging, an atomic operation will be performed in serialization, which causes a huge communication overhead. To address this problem, P-Reduce adopts a new approximate synchronization method; that is, it does not consider the synchronization of node pairs separately but uses a parameter averaging matrix to treat conflicting synchronization operations as one operation. In this way, the existing All-Reduce-related library (Chen et al., 2015; Sergeev and Del Balso, 2018) can also be used. It also does not affect the convergence speed. Nevertheless, partial All-Reduce is associated with several limitations that can impede its effectiveness. Firstly, the model states that on different GPUs may not be completely consistent, which can lead to unstable training and necessitate additional synchronization or compensation

mechanisms to ensure convergence. Furthermore, the presence of bugs can be challenging to reproduce or locate, as the differences caused by local synchronization may not be readily observable. The use of partial synchronization can result in suboptimal final performance, particularly for dense models, where partial synchronization may slow down the training process. This is because the incomplete synchronization of model states can lead to inconsistent updates, which can negatively impact the overall convergence and accuracy of the model.

(4) Zero Redundancy Optimizer (ZeRO) (Rajbhandari et al., 2020). While training a model with billions of parameters is a formidable challenge, scaling it further to a model with trillions of parameters poses an even greater hurdle. The current data and model parallelism still have certain limitations for a device with limited memory. Zero was introduced, which optimizes memory usage and significantly accelerates training speeds while maintaining model efficiency. ZeRO can eliminate redundant memory in data and model parallelism while retaining low communication overhead and high computing efficiency, allowing us to efficiently expand the model using a certain number of devices proportionally. In the analysis of the training model of the existing system, it is found that there are two main aspects of memory consumption: (1) the model state (model state), including optimizer state, gradient, and parameters, like in Fig. 5; (2) the main part of the remaining memory consumption, which is collectively referred to as redundant state, including activation, temporary memory, and uncommonly used fragmented memory. ZeRO will concurrently enhance computing efficiency, communication efficiency, and memory optimization, resulting in a significant performance improvement. To address problem 1, optimizing model state memory, ZeRO developed ZeRO-DP (ZeRO-powered data parallelism). ZeRO-DP eliminates memory state redundancy while maintaining efficient communication through partitioning. This optimization consists of three stages, corresponding to optimizer slices, gradient slices, and cumulative parameter slices. To tackle problem 2, ZeRO introduced ZeRO-R, which optimizes residual state memory. ZeRO-R optimizes redundant memory, encompassing activations, temporary buffers, and infrequently used memory fragments. However, ZeRO is associated with several limitations that can impact its effectiveness. Firstly, ZeRO incurs high communication overhead, which can lead to significant performance degradation. Additionally, ZeRO-2 and ZeRO-3 require frequent synchronization of gradients, parameters, and optimizer states across multiple GPUs, which can result in additional communication overhead and slow down the training process. Moreover, ZeRO is prone to memory fragmentation and parameter partitioning issues, particularly during the gather and release phases, which can lead to memory inefficiencies. The repeated release of memory can also result in memory fragmentation, leading to reduced performance and increased memory usage.

(5) ZeRO-Offload (Ren et al., 2021). The current LLM training usually requires expensive GPU clusters. ZeRo-Offload changes the landscape and wants each user to complete the training of LLMs as much as possible. It can train more than 13 billion parameters on a single GPU. ZeRO-Offload can also be used on multiple GPUs. In addition, it can also be combined with the model parallel to train LLMs. In the research, it is found that many algorithms only use CPU memory without utilizing CPU computation, and ZeRO-Offload significantly reduces communication overhead between GPUs. ZeRO-Offload enables the offloading of calculations to a GPU for LLM training, while simultaneously utilizing CPU memory and calculations for offloading, resulting in greater efficiency and performance. ZeRO-Offload makes the gradient, optimizer status, and the calculation of the optimizer offloaded to the GPU, and simultaneously, parameter calculations, forward propagation, and backward propagation are handled by the CPU, effectively combining an optimal offload strategy with ZeRO-DP. However, ZeRO-Offload is beset by several limitations, including the complexity of offload operation scheduling, which can lead to intricate synchronization strategies between CPU and GPU, ultimately resulting in increased latency. Furthermore, the dispersion of parameters and states across the GPU and CPU can render debugging a challenging task. The optimizer state and gradient are cached in the CPU, necessitating substantial host RAM support. The data transmission between the CPU and GPU may become a bottleneck, heavily reliant on the PCIe or NVLink interconnect technology.

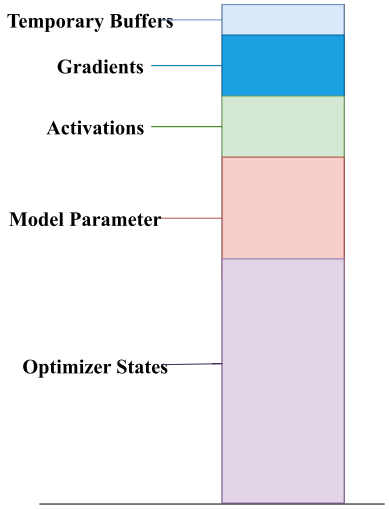


Fig. 5. GPU proportion.

(6) ZeRO-Infinity (Rajbhandari et al., 2021). In recent years, the growth rate of GPU memory has been unable to match the growth rate of model size, and we are getting closer and closer to the GPU memory boundary. The GPU cluster used to train models is out of reach for many data scientists, and even the method of combining model parallelism will increase the burden of reconstructing the model for data scientists. Researchers have thus leaped to 3D parallelism and proposed ZeRO-Infinity to solve this problem. ZeRO-Infinity is a brand-new hybrid system that weighs GPU, CPU, and NVMe memory and uses them to train exceptional models with limited resources. ZeRO-Infinity introduces two innovative techniques to optimize memory usage: the Infinity Offload Engine and Memory-Centric Tiling. The Infinity Offload Engine is a hybrid memory method that dynamically offloads split model states to either CPU or NVMe memory or leaves them in GPU memory, depending on the specific memory requirements. This approach enables efficient memory utilization and reduces the strain on GPU memory. Meanwhile, memory-centric tiling is a GPU memory refinement method designed to minimize the need for working memory during large model training. By leveraging the data acquisition and release mechanism of ZeRO-Stage 3, memory-centric tiling divides the partitioning of massive operators into smaller, manageable segments, which can be applied sequentially to reduce working memory requirements. Nevertheless, the IO operations of ZeRO-Infinity are constrained by the significant latency gap between NVMe storage and memory access, as NVMe I/O latency is substantially higher than that of memory. To mitigate this issue, high-speed NVMe SSDs are required, since frequent swaps can cause training crashes or significantly slow down the training process when lower-speed NVMe SSDs are used. Furthermore, intelligent management of asynchronous scheduling between GPU, CPU, and NVMe storage is essential to optimize performance. Additionally, the initial training phase can be particularly slow due to the large amount of data that must be read from NVMe storage to memory.

(7) Fully sharded data parallelism (FSDP) (Zhao et al., 2023). Large-scale neural network models have grown at an unprecedented rate in recent years. Some recommendation system models can reach

1 trillion parameters. To address the challenge of training a model with trillions of parameters that are hard to load with only one GPU, it is also obtained from DeepSpeed's ZeRO inspiration. FSDP enables the training of LLMs by partitioning model parameters across and utilizing a communication-based strategy to coordinate the parameters. After observation and verification, it should be able to meet the current and near-future requirements. FSDP will model instances (parameters) that are decomposed into small units, and each unit is processed independently. FSDP provides a user-defined fragmentation mechanism, allowing for customized control over the fragmentation process through user-specified functions. Subsequently, all unit and fragment parameters are encapsulated accordingly, ensuring efficient and tailored data processing. To assure reliability, when corresponding calculations (e.g., forward and backward) are performed, the fragmented parameters are restored to complete parameters in peer processes, while other parameters remain fragmented. The memory of peer processes will be released after calculation. Nevertheless, FSDP has several limitations that can impact its performance in distributed deep learning. Firstly, it incurs high communication overhead, which is exacerbated by the frequent and complex communication patterns introduced by parameter sharding. Furthermore, FSDP uses a wrap operation that encapsulates certain submodules of the model into FSDP instances to enable parameter slicing and distributed training. However, the automatic policies governing this wrap operation may not always place FSDP instances optimally, potentially resulting in suboptimal performance. Additionally, FSDP is sensitive to the model's structure, and careful consideration must be given to the placement of the wrap operation to ensure that it does not interfere with the model's functionality or performance.

2.5.2. Model parallelism

A distinctive feature of data parallelism is that each GPU stores a complete replica of the model weights, resulting in redundancy and inefficiencies. To address this limitation, model parallelism offers an alternative approach, where the model is partitioned and distributed across multiple devices. Model parallelism can be further categorized into two subtypes: tensor parallelism and pipeline parallelism. Tensor parallelism enables the decomposition of computational workloads into smaller, independent tasks that can be executed concurrently across multiple processing units, facilitating simultaneous operations such as matrix multiplication and thereby improving performance and efficiency. In contrast, pipeline parallelism allows for concurrent computation across different layers of the model, enabling a more efficient utilization of computational resources.

Pipeline parallelism. The core concept of pipeline parallelism involves breaking down the model into blocks, each corresponding to a specific layer. Different blocks are placed on different devices, e.g., the first layers are set on one device, the middle layers on another device, and the last layers on a third device. During the forward pass, each device propagates the intermediate activation result to the subsequent stage. During the backward pass, each device transmits the gradient result of the input tensor to the previous stage of the pipeline, which enables multiple devices to perform computations at the same time. Pipeline parallel significantly increases training performance. One drawback of pipelined parallelism training is the existence of idle time intervals (bubble time), during which certain devices are not actively involved in computation, resulting in inefficient utilization of computational resources. The communication over the adjacent cut points in the pipelined phase is of the P2P type, where the amount of data in a single communication is small but frequent. Due to the pipeline's characteristics, it creates GPU idle time, which is known as a pipeline bubble.

(1) Naive pipeline. When a model exceeds the capacity of a single GPU, a common approach is to partition the model layers and distribute them across multiple GPUs. While this solution addresses the issue of model size, it introduces new challenges. As illustrated in Fig. 6, one major concern is low GPU utilization: at any given time, only one GPU is actively processing, while the others remain idle. Furthermore, there is no overlap between computation and communication: GPUs are also idle during the transmission of forward-propagated intermediate results or backward-propagated intermediate results. Additionally, this approach results in a high memory footprint: some GPUs must store all activations for a minibatch until the final parameter update, which can be memory-intensive, particularly for large batch sizes.

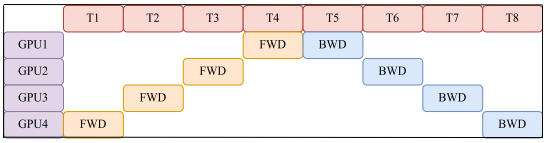


Fig. 6. NaivePipeline.

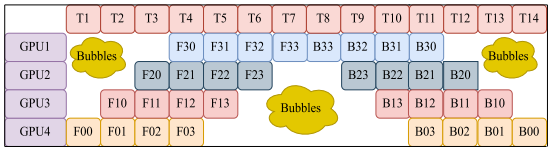


Fig. 7. Gpipe.

(2) GPipe (Huang et al., 2019) is a pipelined parallelism approach that aims to scale the training load of deep learning applications. It achieves this by dividing an L-layer network into K composite layers, each of which is executed on a separate TPU core. Although these composite layers are executed sequentially, GPipe employs a streaming parallelism strategy to mitigate this sequential execution bottleneck. To further improve parallelism, as illustrated in Fig. 7, GPipe subdivides the mini-batch into smaller macro-batches, allowing for increased concurrency. Additionally, GPipe utilizes a simple yet effective recomputation technique to reduce memory requirements, enabling the training of larger models. However, Gpipe still suffers from the presence of bubbles, particularly when the number of microbatches is small. Frequent communication between devices potentially becomes a bottleneck in low-bandwidth scenarios. Additionally, the backpropagation process is inherently serial, which limits the extent of parallelism that can be achieved.

(3) PipeDream (Narayanan et al., 2019) is one of the first projects released by Fiddle that focuses on parallel training of deep learning models. The algorithm flow is as follows. First, PipeDream must coordinate the work of the bidirectional pipeline between different input data. Next, PipeDream manages weight versions within the backward channel to ensure accurate gradient computation and maintain alignment with the forward channel's weight versions. Finally, to achieve optimal pipeline throughput, PipeDream requires all stages within the pipeline to have similar computational durations. Nevertheless, PipeDream is characterized by strong dependencies on microbatches, requiring careful adaptation of the microbatch size to the number of available GPUs. Ensuring consistency in PipeDream training is challenging, as it employs an asynchronous SGD approximation to facilitate the parallelization of the training process.

Tensor parallelism. In contrast to pipelined parallelism, tensor parallelism is an intra-layer partitioning technique, where a tensor is split into N segments along a specific dimension. Each device stores only $1/N$ of the entire tensor, necessitating additional communication to ensure accurate results. This approach can also be viewed as distributing matrix operations across multiple devices, such as dividing a matrix multiplication into smaller matrix multiplications and assigning them to different devices. Communication occurs at each layer during both forward and backward propagation, involving all-reduce operations that exchange large amounts of data frequently.

(1) 1D tensor parallelism. Tensor parallelization is a technique that distributes model parameters across multiple devices to alleviate memory constraints. Megatron-LM (Narayanan et al., 2021) presents an optimized implementation of one-dimensional tensor parallelization, which efficiently partitions model parameters to reduce memory load. Take a linear layer as an example; it involves a $Y = XA$ operation. With two processors, we divide A into $[A_1\ A_2]$ and compute $Y_i = XA_i$ on each processor. Then, we combine the results to form $[Y_1, Y_2] = [XA_1, XA_2]$, known as a columnar-parallel approach. When the second linear layer $Z = Y\ B$ follows the above column-parallel layer, B can be partitioned as $\begin{bmatrix} B_1 \\ B_2 \end{bmatrix}$. This type of parallelism is known as row parallelism. To calculate $Z = \begin{bmatrix} Y_1 & Y_2 \end{bmatrix} \times \begin{bmatrix} B_1 \\ B_2 \end{bmatrix}$, we first compute $Y_i B_i$ on each processor and then use an all-reduce operation to summarize the results as $Z = Y_1 B_1 + Y_2 B_2$. However, 1D tensor parallelism is constrained by the dimensionality of the matrices, which limits the maximum number of GPUs that can be supported. Its applicability is restricted to certain layers; for instance, layer normalization and residual connections require copying all data.

(2) 2D tensor parallelism. The 1D tensor parallelism has a limitation in that it does not partition the activations, which can lead to significant memory consumption for LLMs. To address this issue and achieve a more balanced distribution of computational and memory load, 2D tensor parallelism is introduced, building on the SUMMA algorithm (Schatz et al., 2012). This approach enables the partitioning of both model parameters and activations, allowing for more efficient use of memory and computational resources. Take $Y = XA$ as an example (Xu and You, 2023). Given $P = q \times q$ processors (necessary conditions), and assuming $q = 2$, the input X and weight A can be partitioned into 2×2 blocks. For $\begin{bmatrix} X_{00} & X_{01} \\ X_{10} & X_{11} \end{bmatrix}$ and $\begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix}$, the computation is divided into q steps, in this case.

When $t = 1$, X_{i0} is propagated in its rows and A_{0j} is propagated in its columns.

Then multiply X_{i0} and A_{0j} on each processor (i, j) as.

$$\begin{bmatrix} X_{00}A_{00} & X_{00}A_{01} \\ X_{10}A_{00} & X_{10}A_{01} \end{bmatrix} \tag{1}$$

When $t = 2$, X_{i1} is propagated in its rows and A_{1j} is propagated in its columns, multiplying them as:

$$\begin{bmatrix} X_{01}A_{10} & X_{01}A_{11} \\ X_{11}A_{10} & X_{11}A_{11} \end{bmatrix} \tag{2}$$

By adding (1) and (2), there is:

$$Y = XA = \begin{bmatrix} X_{00}A_{00} + X_{01}A_{10} & X_{00}A_{01} + X_{01}A_{11} \\ X_{10}A_{00} + X_{11}A_{10} & X_{10}A_{01} + X_{11}A_{11} \end{bmatrix}$$

Nevertheless, 2D tensor parallelism incurs higher communication overhead compared to 1D, due to the introduction of an additional communication dimension. This increased complexity necessitates the use of all-gather and reduce-scatter operation.

(3) 2.5D tensor parallelism. Relative to 1D tensor parallelism, 2D tensor parallelism reduces memory usage, but may introduce more communication costs. Therefore, 2.5D tensor parallelism is proposed based on 2.5D SUMMA (Schatz et al., 2012), which reduces communication by using more devices. The example of $Y = XA$ remains. Given $P = q \times q \times d$ processors (necessary conditions), and assuming that $q = d = 2$, the input X can be partitioned into matrices of $d \times q$ rows and q columns as follows (Wang et al., 2022b): $\begin{bmatrix} X_{00} & X_{01} \\ X_{10} & X_{11} \\ X_{20} & X_{21} \\ X_{30} & X_{31} \end{bmatrix}$

It can be reshaped as a d-layer:

$\begin{bmatrix} X_{00} & X_{01} \\ X_{10} & X_{11} \end{bmatrix}$ *and* $\begin{bmatrix} A_{20} & A_{21} \\ A_{30} & A_{31} \end{bmatrix}$

The weight A reshape is: $\begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix}$

then

$$Y = XA = \text{concat}(\begin{bmatrix} X_{00} & X_{01} \\ X_{10} & X_{11} \end{bmatrix}\begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix}, \begin{bmatrix} X_{20} & X_{21} \\ X_{30} & X_{31} \end{bmatrix}\begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix})$$

Based on the above derivation, it can be found that the two matrices being spliced can be naturally computed in parallel. $\begin{bmatrix} X_{00} & X_{01} \\ X_{10} & X_{11} \end{bmatrix}\begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix}$ *and* $\begin{bmatrix} X_{20} & X_{21} \\ X_{30} & X_{31} \end{bmatrix}\begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix}$

Notably, the two matrix multiplications mentioned above are in a form that is directly applicable to the 2D tensor parallelism technique described earlier. Specifically, a total of eight processors are utilized, with each group of four processors employing 2D tensor parallelism to perform the corresponding matrix multiplications. The results of these two parallelisms are then simply concatenated to obtain the final output.

(4) 3D tensor parallelism aims to parallelize the computation of a neural network model with the expectation of obtaining the best communication cost optimization. Let us take the computation of the linear layer $Y = XA$ as an example. Given $P = q \times q \times q$ processors (necessary conditions), such as $q = 2$, divide the input X and weights A into $\begin{bmatrix} X_{000} & X_{001} \\ X_{010} & X_{011} \\ X_{100} & X_{101} \\ X_{110} & X_{111} \end{bmatrix}$ *and* $\begin{bmatrix} A_{000} & A_{001} & A_{010} & A_{011} \\ A_{100} & A_{101} & A_{110} & A_{111} \end{bmatrix}$ where each X_{ijl} and A_{ijl} is stored on process (i, j, l), and then X_{ijl} is collected on $(i, 0.....q, l)$ on which X_{ijl} is collected, and on $(0.....q, j, l)$ to collect A_{lji}. Thus, it is possible to have X_{il} and A_{lj} on each processor (i, j, l) having obtained $X_{il}A_{lj}$, and finally on $(i, j, 0.....q)$ on $(i, j, 0..q)$ reduce-scatter the result to get Y_{ijl} $\begin{bmatrix} Y_{000} & Y_{001} \\ Y_{010} & Y_{011} \\ Y_{100} & Y_{101} \\ Y_{110} & Y_{111} \end{bmatrix}$.

However, 2.5D tensor parallelism requires the use of 3D all-reduce and scatter operations, which, despite their complexity, still incur significant communication overhead. If the batch size is not sufficiently large, the 3D splitting can reduce device utilization.

2.5.3. 3D parallel

The rapid growth in computing power available in modern GPU clusters has made training trillion-parameter models a feasible reality, rather than a distant dream. In response, a 3D parallelism solution (Lai et al., 2023) has been proposed to tackle the training challenges of such massive models. This approach combines the strengths of three powerful techniques: data parallelism, model parallelism (tensor parallelism), and pipeline parallelism. By integrating these methods, 3D parallelism can simultaneously address the dual challenges of memory efficiency and compute efficiency when training trillion-parameter models. Building on the success of DeepSpeed (Rasley et al., 2020), which can train most LLMs without sacrificing speed, 3D parallelism leverages the complementary strengths of data, tensor, and pipeline parallelisms to optimize memory and compute efficiency. Specifically, each layer of the memory efficiency model is decomposed into pipeline stages, which are further divided into tensor parallelism units that can be computed efficiently. Moreover, data parallelism can be expanded beyond model and pipeline parallelism, enabling further scalability.

2.6. Comparison between different strategies

To facilitate a more nuanced understanding, we conduct a comparative analysis of the various parallelization strategies discussed earlier, including data parallelism, model parallelism, in this section. This comparison aims to provide a critical insight into the strengths and weaknesses of each approach, and compare the disparate implementation methodologies employed by each of them.

Table 4
The development of LLM.

Year	Model	Description
2014	Attention (Luong et al., 2015)	Attention is the cornerstone of LLM at present. It was originally proposed in the field of machine translation.
2017	Transformer (Vaswani et al., 2017)	Transformer proposed a new neural network structure based on the self-attention mechanism, completely abandoned CNN and RNN. At the cost of averaging the weighted position of attention and reducing the effective resolution, a certain number of operations are reduced, which use multi-head attention to offset.
2018	BERT (Devlin et al., 2018)	BERT is a representative of this type of model, which is encoder-only. It uses the transformer structure and pre-training are performed on a large text corpus.
2020	ChatGPT-3 (Brown et al., 2020)	ChatGPT is a cutting-edge dialogue system developed by OpenAI that is based on large-scale language models, which is a powerful deep-learning model.

In the context of data parallelism, we can distinguish between two primary approaches: data parallelism (DP) and distributed data parallelism (DDP) (Li et al., 2014; Gibiansky, 2017; Luo et al., 2020). DP is characterized by its simplicity and ease of implementation, as well as its fast communication speed, making it an attractive option for certain applications. However, its scalability is severely limited by the constraints of single-machine GPU memory and computing resources, rendering it less suitable for large-scale computations that require significant parallel processing capabilities. DDP offers a highly scalable architecture, enabling effortless expansion to multiple machines and demonstrating strong compatibility with various computing environments. Nevertheless, DDP is not without its limitations, as it incurs significant network bandwidth overhead, which can result in substantial communication costs. Furthermore, DDP's performance is constrained by the synchronization communication overhead, particularly when using large batch sizes. In comparison to DDP, DP is limited by the number of GPUs available on a single machine and lacks support for cross-machine utilization. As a result, DP has become less favorable, and is no longer recommended. In contrast, DDP has emerged as the officially endorsed method due to its efficient communication protocol and seamless scalability to large-scale, multi-machine systems.

In model parallelism, a nuanced distinction can be drawn between two predominant approaches: pipeline parallelism and tensor parallelism. These two paradigms differ fundamentally in their methodology and application. Pipelining parallelism is a distributed computing paradigm that involves partitioning a deep neural network into a sequence of layers, which are then assigned to distinct GPUs for execution. The pipeline parallelism model (Huang et al., 2019; Narayanan et al., 2019) enables the division of layers and their allocation to different devices, allowing the transmission of activation values between layers. This approach is particularly well-suited for large-scale models with multiple stacked layers, as it facilitates the efficient utilization of computational resources. However, pipeline parallelism can also lead to device underutilization, resulting in idle GPUs, which can negatively impact the overall performance and efficiency of the system. Tensor parallelism (Narayanan et al., 2021; Xu and You, 2023; Schatz et al., 2012) employs a novel approach to parallelize the computation of deep neural networks by partitioning tensors and matrices along specific dimensions at each layer. This enables the synchronous and parallel execution of the partitioned tensors on multiple devices, thereby facilitating the efficient utilization of computational resources. However, this approach necessitates frequent synchronization of intermediate tensor results, which can introduce additional overhead. Despite this, tensor parallelism is particularly well-suited for LLMs characterized by substantial single-layer parameters, as it can effectively leverage the parallel processing capabilities of multiple devices. Furthermore, the use of GPUs is often more efficient in this context. A comparative analysis of pipeline parallelism and tensor parallelism reveals distinct differences in their parallelization strategies. Specifically, pipeline parallelism targets the model layer as its parallelization units, resulting in a relatively small amount of communication overhead. However, this approach is also prone to GPU idle time, which can negatively affect overall system efficiency. In contrast, tensor parallelism focuses on intra-layer tensors as its parallelization units, which requires a larger amount of communication but yields higher GPU utilization. Notably, both pipeline parallel and tensor parallel exhibit strong composability.

2.7. The architecture of LLM

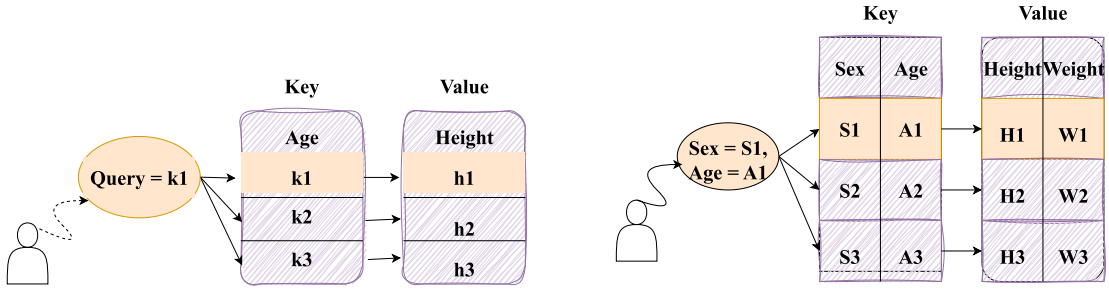
This part introduces some underlying technologies and architectures used in LLM model training, including attention mechanism, Transformer network structure, and the hybrid expert system. Moreover, we introduce different large model styles, including BERT-style, which can be either Encoder–Decoder Only or Encoder-Only; and ChatGPT-style, which is exclusively Decoder-Only (Yang et al., 2024). Here, we summarize the development of LLM in Table 4

2.7.1. Attention

Attention (Bahdanau et al., 2014) is the cornerstone of LLM at present. It was originally proposed in machine translation (Bahdanau et al., 2014). The encoder–decoder method has a potential problem in the neural network, which needs to compress all the necessary information into a fixed-length vector, which will affect performance when processing long inputs. The concept of attention is proposed to solve this problem. The complete sequence will not be converted into a fixed-length vector, but the input sequence will be encoded and decoded in a subset of these vectors. Attention differs from the traditional RNN architecture in that it utilizes a bidirectional RNN. Below, we introduce self-attention (Yang et al., 2016), which is the foundation of Transformer.

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V.$$

To better understand the formula of self-attention, let us first understand a simple example. We have a table of key–value pairs, and we can index the value corresponding to the key in the table by initiating a query. For example, in Fig. 8(a), we have age as the key and the average height of the age as the value. We want to find the corresponding key through the query to obtain the value we need. Then, with another example, suppose we have a key–value pair with (gender, age) as the key and (height, weight) as the value. As illustrated in Fig. 8(b), we can find the corresponding key through a query (gender, age) and get the value we need. Now we can explain the classic formula above. First, in Fig. 10, suppose we have an input x. We will first embed x, converting it into a high-dimensional vector, and then multiply them by W_q, W_k, and W_v — randomly initialized weight matrices — to perform a linear transformation. This results in the matrices Q (query),



(a) Example 1. (b) Example 2.

Fig. 8. Example of Q, K, and V.

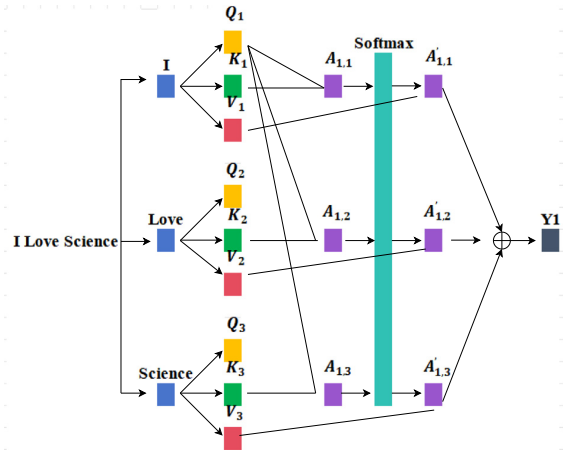


Fig. 9. Self-attention process.

K (key), V (value), which can be interpreted as retrieving an output based on a query within a key–value pair structure. There are two most commonly used implementation methods of self-attention: one is based on accumulation, and the other is based on the dot product. Because dot products actually can use the optimized matrix multiplication algorithm, which is faster and space-saving, Transformer's self-attention is based on dot products. To give a specific example, like in Fig. 9, suppose we have the input of "I love science". First, the input is embedded, and then it is linearly transformed to obtain the matrices Q, K, and V. The column vector q_1 in Q will be multiplied by k_1, k_2, and k_3, respectively (k_1, k_2, and k_3 are divided from K). To get our attention score, a_1, a_2, and a_3 are normalized by softmax(a_1, a_2, a_3) to get a'_1, a'_2, and $a_3\prime$ multiplied by v_1, v_2, and v_3 respectively and accumulated to get our final output (v_1, v_2, and v_3 are divided from V). By abstracting it, it can be transformed into a matrix calculation, back to Fig. 10. Then we do the dot product of K^T and Q according to the formula, and get the matrix A (attention-score matrix, concat by a_1, a_2, and a_3). Dividing A by the coefficient d_k. d_k is processed here to ensure the stability of the softmax(A) gradient. Otherwise, the value after softmax may be very small. Then we perform softmax normalization to get A', and finally multiply A' by V to get our attention score. It also has various forms, like hard attention (Xu et al., 2015), local attention (Luong et al., 2015), co-attention (Lu et al., 2016), etc.

2.7.2. Transformer

In the past, encoder–decoder models with attention mechanisms have demonstrated good performance. Transformer proposed a new neural network structure based on the self-attention mechanism. Transformer (Vaswani et al., 2017) completely abandoned the convolutional neural networks (CNNs) and recurrent neural networks (RNNs). At the cost of averaging the weighted position of attention and reducing the effective resolution, a certain number of operations are reduced. This utilizes multi-head attention to offset. Transformer is the first representation that fully uses self-attention to calculate input and output without using an aligned RNN or a convolution model. The Transformer's encoder consists of multiple identical layers, as shown in Fig. 11(a). Each layer contains two sub-structures: the multi-head self-attention layer and a fully connected feedforward neural network layer. Similarly, the decoder is composed of multiple identical layers, but each includes three subcomponents: the two found in the encoder, plus an additional masked multi-head attention layer. Here we introduce the processing details. The transformer uses a self-attention mechanism, which is divided into an encoder and a decoder. When processing the input, the transformer will add position information after the input embedding, because the Transformer uses global information and cannot use the order information of the words.

Each layer of the encoder possesses two sub-parts: multi-head attention and feed-forward layers. Multi-head attention can be considered an enhancement of self-attention. It will further improve the Q, K, and V matrices of self-attention. Dividing into sub-matrices, like in Fig. 11(b), it will perform self-attention calculations in parallel and then concatenate them together. Next, multiple-head attention performs a linear transformation to obtain the final output Z, whose dimensions are consistent with the dimensions of the input x. Then perform *Add* and *Norm*. *Add* is to perform a residual connection to prevent network degradation. *Norm* is to perform a layer normalization on Z to ensure an even distribution of results. The normalized output then serves as input for the feed-forward layer. The operation of *Add* and *Norm* is repeated until the end.

The Decoder has three sub-layers (Vaswani et al., 2017). In addition to multi-head attention and feedforward, there is also masked multi-head attention. The meaning of "masked" here in practical applications is to predict based on all the information before i when predicting the ith word, and the information behind i is "masked" and will not be used. In actual processing, just set the following attention score to 0. After processing, the output undergoes *Add* and *Norm*, followed by a combination with the Encoder's result using multi-head attention. This is succeeded by another *Add* and *Norm*. As the input for feed-forward, it is processed, and then *Add* and *Norm* are performed. After all processing is completed, the result is linearly transformed, and then softmax is performed to obtain the final output result.

2.7.3. MoE

The mixture of experts (MoE) (Gan et al., 2025b) is an ensemble learning technology. When the traditional deep learning model is trained, for each input sample, the entire network will participate in the calculation. As the model becomes larger and larger, more and more sample data are used for training. The training overhead becomes more and more unbearable. MoE (Rajbhandari et al., 2022)

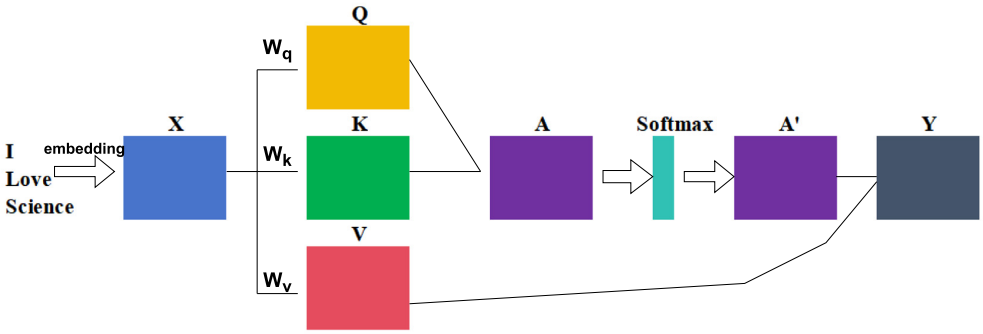
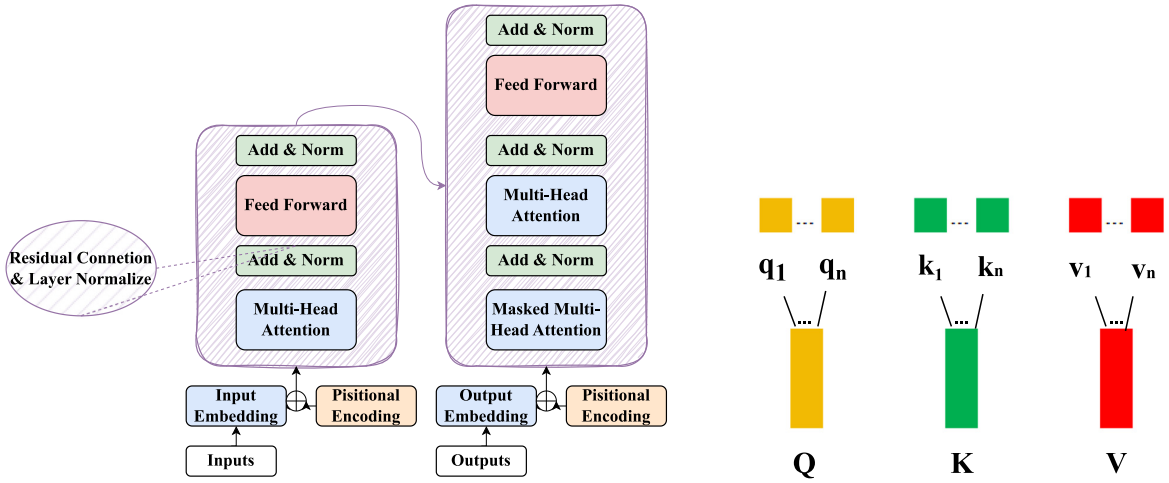


Fig. 10. Self-attention process in matrix.



(a) Transformer (b) Multiple head attention

Fig. 11. The structure of Transformer and multiple-head attention.

can dynamically activate the part of the neural network, thereby greatly increasing the number of model parameters without increasing the amount of calculation. MoE decomposes the predictive modeling task into multiple subtasks, each of which is addressed by a dedicated expert model. A gating model is also developed to determine which expert to trust for each input, based on the input's characteristics. The gating model then combines the predictions from the various experts to produce the final output. In the MoE architecture, a group of experts and a gating model cooperate to solve nonlinear supervised learning problems by partitioning the input space into a nested set of regions. However, conventional MoE architectures may be susceptible to the issue of imbalanced expert load, where certain experts are activated with high frequency, while others remain underutilized. Furthermore, in the context of multi-machine distributed training, the distribution of experts across different GPUs can result in significant cross-device communication overhead, especially when high-frequency expert switching occurs.

2.7.4. Encoder–decoder only or encoder-only

At present, the two most important models in the industry are BERT (Devlin et al., 2018) and ChatGPT (Brown et al., 2020). These models can be broadly classified into two categories: encoder–decoder or encoder-only (namely BERT-like) and decoder-only (namely GPT-like). The following introduces these two types. In the early stage of LLM development, encoder-only or encoder–decoder models are more popular, especially after BERT has achieved explosive growth. BERT (Devlin et al., 2018) is a representative of this type of model, which is encoder-only. It uses the transformer structure and pre-training performed on a large text corpus. The architecture of the previous pre-training models is constrained by the unidirectional nature of the language model (either left-to-right or right-to-left), which consequently restricts the model's capacity for representation, making it only obtain unidirectional context information. BERT fuses left and right context information for deep bidirectional language representation. BERT is a pre-trained language representation model that leverages LLM (Devlin et al., 2018) to generate comprehensive bidirectional language representations. BERT stacks Transformers to form a deeper neural network. BERT is currently an important processing tool in the NLP field. The BERT-like model also includes RoBERTa (Liu et al., 2019), T5 (Raffel et al., 2020), etc.

2.7.5. Decoder only

ChatGPT (Brown et al., 2020) is a cutting-edge dialogue system developed by OpenAI that is based on large-scale language models. It is a powerful deep learning model. ChatGPT's training utilizes a vast amount of text data, including diverse texts sourced from the internet. Through this large-scale training, the model can learn the underlying patterns, rules, and semantic associations of language, so that it can

generate plausible and coherent responses. The goal of ChatGPT is to have a natural and smooth conversation with users, providing useful information and help. It can answer various questions, provide explanations and suggestions, and have some interesting conversations. Whether it is science, history, culture, or any other topic, technical or not, ChatGPT strives to provide accurate and helpful answers. While ChatGPT is powerful in many ways, it has some limitations. Since it is trained on historical text data, it may be influenced by biases, inaccuracies, or outdated information present in the data. In addition, ChatGPT lacks true common-sense reasoning and understanding, which can limit its effectiveness in addressing complex problems or situations that require deep reasoning. To ensure safe and responsible use, ChatGPT from OpenAI is subject to certain limitations and vetting processes. However, despite these efforts, there may still be potential risks and inaccuracies. Therefore, always exercise caution when using the answers and suggestions provided by ChatGPT, and verify them yourself if necessary. In general, ChatGPT is a powerful dialogue system designed to have a natural and fluent dialogue with users and provide useful information. It is a technology based on a large-scale language model that can answer various questions and help in many areas. The GPT-like models also includes OPT (Zhang et al., 2022), PaLM (Chowdhery et al., 2022), BLOOM (Scao et al., 2022), and CodeX (Godoy et al., 2023), etc.

2.8. Optimized method

This part will summarize some optimization techniques used in LLMs. The main optimization techniques used include model compression, activation-related optimization, mixed precision, optimizers, etc. Model compression is a common strategy. By reducing the number of parameters to reduce computation and storage requirements. Reducing activation computation is another strategy that speeds up training by reducing redundant computation of LLMs. Mixed-precision computation exploits changes in floating-point precision to improve computation efficiency. The optimizer strategy involves selecting appropriate optimization algorithms and hyperparameter settings to accelerate convergence and minimize training time, as introduced below.

2.8.1. Model compression

The daunting size and computing requirements of current LLMs are a huge challenge for deployment with limited resources, such as deployment on embedded devices. Model compression can convert large, resource-intensive models into compact versions suitable for storage on constrained devices. Additionally, it can optimize the model to execute faster with minimal latency or achieve a balance between these goals. LLM may also cause environmental problems. Limited resources may become a barrier to obtaining the basic hardware required for models, and LLMs' large energy consumption will exacerbate carbon emissions. Artificial intelligence (AI) research and sustainable development are also very important issues. One possible solution to these challenges is to utilize model compression techniques. Under the premise of maintaining a certain performance of LLM, the use of model compression can reduce carbon emissions. Currently, model compression mainly consists of pruning, knowledge distillation, quantization, and low-rank factorization (Cheng et al., 2017; Zhu et al., 2024a).

Pruning is an effective model compression technique that reduces model size and complexity. Many redundant parameters in LLMs have minimal impact on model performance (Li et al., 2016). Therefore, discarding these parameters can compress the model while retaining LLM performance to the maximum extent (Ardakani et al., 2018; Fang et al., 2023). Pruning can be divided into unstructured pruning (Zhang et al., 2018) and structured pruning (Anwar et al., 2017). The difference between them lies in the target of pruning and the resulting network structure. Structural pruning removes elements of the entire structure, including neurons, channels, and layers. This reduces model complexity and memory footprint while maintaining the overall structure of the model, such as LLM-Pruner (Ma et al., 2023). Unstructured pruning will remove specific parameters without considering the internal structure of the model. A threshold will be set during pruning, and parameters that do not reach this threshold will be set to zero and then pruned. However, this can make the model unstable and irregular. Sparse model structures resulting from this approach often require retraining to regain model accuracy. In contrast, SparseGPT (Frantar and Alistarh, 2023) and RAPrune (Zhang et al., 2023) do not require retraining and combine efficient parameter tuning.

Knowledge distillation is an effective machine-learning method to improve model performance and generalization. The core idea of knowledge distillation (KD) is to transform teacher models with complex knowledge into more streamlined and efficient representations. Whether the model distills the emergent ability (EA) (Schaeffer et al., 2023) of an LLM into a smaller model can be categorized into standard KD and EA-based KD. Standard KD enables the student model to learn the teacher model by mimicking its output distribution and feature representations. EA-based KD not only distills the common knowledge of LLM into student models but also transforms their EA around them. EA is composed of several interesting facts, including in-context learning (Dong et al., 2023; Akyürek et al., 2022), chain-of-thought (CoT) (Wei et al., 2022; Wang et al., 2022a), and instruction following (Brooks et al., 2023).

Quantization is a widely used technique that can alleviate the storage and computing overhead of deep learning models (Gholami et al., 2022). Commonly, traditional representations use floating-point numbers. Quantization converts these representations into integers or discrete forms, significantly reducing storage requirements and computational complexity. This can be split into three main methods: quantization-aware training (QAT) (Kim et al., 2022), quantization-aware fine-tuning (QAF) (Dong et al., 2019), and post-training quantization (PTQ) (Liu et al., 2021). QAT is to seamlessly integrate quantization into the model training process, using low-precision representation in training, and enhance the model's ability to handle the accuracy loss caused by quantization. QAF adds quantification to the process of model fine-tuning, where the main goal is to ensure that the performance of LLM is maintained during the fine-tuning process. PTQ quantifies the parameters of the model after LLM training is completed. The main purpose is to reduce the complexity of model storage and calculation, and there is no need to modify the model structure or re-train. It is crucial to acknowledge that the quantization procedure involved in PTQ can result in a certain level of reduction in precision.

Low-rank factorization is a model compression technique that decomposes a given weight matrix into two smaller matrices and performs approximate calculations. Low-rank factorization can significantly reduce the dimension of the matrix, such as decomposing a matrix A into two matrices W and V, if the approximate calculation results of W and V product can approximate the original matrix. Low-rank factorization can dramatically reduce the parameter quantity and calculation overhead (Idelbayev and Carreira-Perpinán, 2020).

LoRA and QLoRA (Dettmers et al., 2023) are two efficient fine-tuning techniques designed for LLMs, significantly reducing computational and storage overhead while enabling seamless adaptation to distributed training and inference scenarios. By minimizing the communication volume between nodes in a distributed environment, these techniques facilitate more efficient collaboration. During distributed training or inference, each node computes the LoRA gradient, and AllReduce synchronization consolidates the updates. However, varying computational complexities across different LoRA adapters can lead to fluctuations in node utilization.

2.8.2. Activation

The optimization for activation is mainly to reduce the memory consumption of some redundant parameters or model states during the calculation of activation. The following introduces checkpointing and its improved version, which is ‘‘reduce activation’’.

Checkpointing (Chen et al., 2016). In recent years, the model has become larger and larger, and the memory of devices limits the training process. Therefore, there is a need for a memory-efficient training algorithm. When we are training the model, there is usually a large share of memory for intermediate results and gradient storage. Checkpointing (Chen et al., 2016) mainly reduces the memory consumption of intermediate results and gradients. The algorithm only stores the output results of each segment in the forward propagation and discards the intermediate results of all segments. The discarded intermediate results are propagated backward and recalculated from the segment level. In essence, the algorithm stores most activations and recomputes them when calculating gradients in back-propagation.

Reduce activation (Korthikanti et al., 2023). Computational issues are a major challenge in training LLMs. Activation recomputation, compared to directly storing activation for back-propagation, only performs repeated calculations, which saves memory but adds additional redundant calculations. Therefore, some studies have proposed that reducing activation speeds up the training of the model. Two new techniques are proposed here, sequence parallelism and selective activation recomputation, to diminish the memory pressure of storing activation and the demand for recomputed activation. *Sequence parallelism*, which operates along the sequence dimension, reduces the memory requirements for activations. For selective activation recomputation, checkpointing and recomputation of only specific parts have significant memory consumption but lower computational demands.

2.8.3. Mixed precision

Increasing the size of the model also improves its accuracy, but increases the memory and computing requirements when training the model. These requirements can be expressed by reducing the accuracy, the pressure on memory bandwidth, calculation time, and mixed precision (Narang et al., 2018) using three techniques to maintain the accuracy of the model. Mixed precision maintains a master copy of weights in FP32 for loss scaling to minimize gradient values becoming zeros, and FP16 for calculation and accumulation into FP32. *FP32 copy of weight* uses FP16 for calculation in forward and backward propagation. Compared with the calculation that requires FP32, the memory and bandwidth are directly halved. However, some parameters represented in FP16 in some update processes will become 0, so an FP32 copy is needed to recover at this time to maintain accuracy. On *Loss scaling,* as FP16 will have some exponential bias, an effective way to change the gradient value to FP16 representation is to calculate the scale of the loss value in the forward propagation, which can ensure that the gradient value is scaled by the same amount. On *arithmetic precision*, there are mainly three types of calculations in large neural networks: vector dot product, reduction, and point-wise operation. To maintain the accuracy of the model, it is essential to accumulate the partial product of the FP16 vector dot product into an FP32 value and convert it into FP16 before writing it into memory. Large reductions should use FP32 to perform calculations. The accuracy of point-wise operation calculation does not affect the calculation speed; both FP16 and FP32 can be used. In summary, FP16 is used to store and calculate weights, activations, and gradients, while the FP32 master copy of weights is used for updating.

2.8.4. Optimizer

The optimizer plays a crucial role in the back-propagation process by correctly adjusting the model's parameters to minimize the loss function and optimize its performance. The optimizer continuously iteratively optimizes the parameters (Ruder, 2016). The optimization algorithm based on gradient descent (GD) is extremely important in the fields of scientific research and engineering. GD is a powerful optimization technique that can effectively solve complex optimization problems with large datasets and high feature dimensions. The early ones include batch gradient descent (BGD) (Wang et al., 2020), which uses all samples in each iteration. The convergence process of the loss function is stable, but the calculation is slow, and the data cannot be dynamically updated. Stochastic gradient descent (SGD) (Schaul et al., 2013) selects a part of the samples for each update and converges quickly, but the convergence process fluctuates too much. Mini-batch gradient descent (MBGD) (Khirirat et al., 2017) selects a certain number of original updates for each update, striking a balance between training speed and convergence efficiency. However, convergence cannot be guaranteed, and oscillations may occur, while the learning rate remains fixed. Momentum (Qian, 1999) introduces the idea of momentum to accelerate convergence and reduce oscillation. Subsequently, an algorithm based on adaptive learning rate emerged, allowing the learning rate to change dynamically. An example is Adaptive Gradient (AdaGrad) (Duchi et al., 2011), which uses adaptive learning rates and requires no artificial changes, although a global learning rate must be set. Another method, AdaDelta Zeiler, 2012, does not rely on a global learning rate. The acceleration effect is good in the early stage of training, but it may oscillate at the local minimum later. RMSProp (Tieleman and Hinton, 2017), a development of AdaGrad, modified AdaGrad's sum of squared gradients to be an exponentially weighted average. ADAM (Kingma and Ba, 2014) combines the two algorithms of Adagrad and RMSProp, integrating their advantages. It is a highly efficient stochastic optimization technique that solely relies on first-order gradients and consumes minimal memory. It comprehensively incorporates the estimation of both first-order and second-order moments of the gradient. It can effectively accelerate convergence and reduce training time. NADAM (Dozat, 2016), based on ADAM, adds Nesterov.

3. Key challenges

With advancements in model training technology, the models we train are becoming increasingly larger and more accurate, showing significant performance improvements. It feels as though we are not far from the strong AI portrayed in science fiction movies. However, there are still many problems that need to be solved, and the obstacles in our path must be addressed and overcome. Now let us look at the current challenges that LLM training may face:

3.1. Datasets

Datasets play a vital role in LLM training (Alajrami and Aletras, 2022; Kaplan et al., 2020; Zha et al., 2025) and can be roughly divided into pre-training, fine-tuning datasets, etc. In addition, datasets may also have privacy issues and come from a wide range of sources, including papers, web articles, and books. Obtaining good, high-quality, and accurate datasets requires a significant amount of resources (Du et al., 2022; Qiu et al., 2022).

Pretraining dataset. The quality of the pre-training dataset is closely related to the performance of LLM (Du et al., 2022; Qiu et al., 2022). It can fundamentally determine the quality and performance of LLM. It can help LLM better and more diversely understand our real world, including our language, culture, values, etc. LLM training usually requires obtaining a large number of datasets, but obtaining high-quality, large-scale datasets is an important challenge.

Fine-tuning overhead. In many cases, the amount of data required for fine-tuning may be similar to the amount of pre-training data, so in many cases, we may not necessarily have enough data for fine-tuning. We can roughly divide it into the following three cases: zero-shot (Yin et al., 2019), few-shot (Brown et al., 2020), and abundant-shot. In the case of zero-shot, the best way is not to perform fine-tuning; directly using LLM will be a better choice (Yin et al., 2019). In the few-shot scenario, it is recommended to use the in-context method (Lee et al., 2022; Ruder et al., 2019) for LLM fine-tuning, which can even match the state-of-the-art fine-tuning models. However, in the case of abundant data, the choice becomes more flexible, and whether to fine-tune or not depends on other factors related to the data, ultimately aiming to choose the most effective approach.

Privacy. When we have sufficient datasets, we may also encounter some privacy issues (McCallum, 2022). For example, the data needs to be desensitized to a certain extent; otherwise, the privacy of other people may be leaked, such as the information of patients in hospitals and bank users' deposits, etc. Some outlaws obtain sensitive privacy data through some means in cyberspace. Threat modeling (Xiong and Lagerström, 2019) obtains some sensitive assets of LLM users (such as training sets, model parameters, etc.) through certain means. There are also privacy attacks, such as member inference attacks (Hu et al., 2022) that use some non-sensitive information of the model to infer user-sensitive information in the model. Reconstruction attacks, as studied in Garfinkel et al. (2018), aim to reconstruct multiple training samples and their corresponding labels, to restore confidential characteristics or fill in missing data instances. Attribute inference attack (Zhao et al., 2021) refers to using publicly visible attributes and structures to infer hidden or incomplete attribute data. Model extraction (Truong et al., 2021) is a form of black-box attack where the attacker aims to retrieve information and potentially reconstruct the target model by creating a surrogate model that exhibits similar behavior to the model under attack.

3.2. Efficiency

Cost. In LLM training, we need to use huge resources, including time, money, electricity, etc. For example, GPT-3 has eight different sizes, with parameters ranging from 125M to 175B, and the expense of AI is skyrocketing. The cost of training GPT-3 reached $4.6M. The size of SOTA (state-of-the-art) language models experiences at least a tenfold growth rate each year (Li, 2020b). Training an LLM with total energy amounted to a staggering 13.8 MWh. This model was not being trained to a complete version, but only until 13%. A full training of it will run sixty or even more days. According to the statistics, the total energy consumption to train a complete version of it would be approximately $(60/8) \times 13.8 = 103.5$ MWh, or 103,500 kWh (Dodge et al., 2022). How to train the model in a resource-friendly way is a challenge.

Latency. Latency is also a problem we encounter when using LLM. The API call time varies unpredictably, significantly impacting the user experience. Users do not want a large delay during the use process (Pungas, 2023). The time taken for a large model to call the API of a completion request is mostly influenced by two factors (openAI, 2023): the specific model used and the quantity of generated tokens. Requesting a large amount of generated token completions can lead to increased latencies, and lower max tokens can reduce the latency when we use the API of LLM. The technique includes stop sequences and generates fewer completions.

3.3. Discrimination

The use of pre-trained data in the in-context learning process for LLM training may raise concerns about discrimination, as the model may learn biases from the pre-existing data. The root cause is that the dataset may contain some discriminatory problems. In fact, in the past, machine learning also encountered discriminatory issues during the development stage of learning, such as gender, race, occupation, age, etc (Dong et al., 2023). For example, based on past data, the model will be biased towards selecting men for certain jobs even though the job may have nothing to do with gender; or banks tend to lend money to people of specific colors, and the model learns this biased data during the training process, which will cause the model to have biased descriptions of some occupations and jobs. As the scale of the model increases, the problem of discrimination may become more and more serious. Currently, we do not have an exact indicator to quantify the fairness of LLM. Discrimination exists objectively in the model. We may be able to mitigate the discriminatory effects of the model by fine-tuning the LLM with non-discriminatory data, thereby enabling the model to learn from human teachings and become a relatively unbiased model.

3.4. Content

Limited context length. The input of LLM is limited, and it is not possible to input excessively long text. This limitation may cause issues in using LLM, especially for long text content or text summaries. For generation tasks in ChatGPT-3.5, the maximum length of input text is typically 4096 tokens. This includes entered conversation history, questions, or instructions, etc. For classification tasks, the maximum length of input text is typically 2048 tokens. This includes text content to be classified. For summarization tasks, the maximum length of input text is typically 2048 tokens. This includes the original text that needs to be summarized. Thus long input needs to be shortened or summarized to fit within the model's limits. This may involve truncating the text or reducing its complexity to ensure that it can be processed efficiently and accurately.

Hallucination. The content generated by LLM may contain factual inaccuracies—that is, fabrications or hallucinations. They might produce information that does not exist in reality or reference real sources. However, because its output is often highly coherent and logically consistent, it can mislead users who lack relevant background knowledge. Examples include AI robots inventing non-existent books and research, publications falsely attributed to professors, fake academic papers, fake legal citations, non-existent Linux system features, and technology details that make no sense (Hu et al., 2023). As LLM becomes more prevalent in society's daily life in the future, and people's reliance on it grows, the potential harm it may cause will be amplified (Du et al., 2022; Geirhos et al., 2020; Tang et al., 2021; Liang et al., 2023).

Harmful content. LLM will generate different types of harmful content (Tang et al., 2024). Currently, LLM has some restrictions on the output content. It may output content that is inconsistent with human values and output some content that is harmful to individuals, groups, and even society. On occasion, the content generated by LLM may exceed acceptable limits, outputting violent, hateful, discriminatory, or inappropriate content for children, promoting harmful values, and potentially exacerbating the hostile online environment.

3.5. Model evaluation

Currently, it is challenging to conduct a truly objective performance evaluation on numerous models. While these models may perform well in benchmark tests, they often encounter significant issues when applied to real-world industry data (Yang et al., 2024). Therefore, evaluating models on real-world datasets requires more rigorous methods to better assess the effectiveness of the model in practical applications (Chung et al., 2022; Buolamwini and Gebru, 2018; Beukeboom and Burgers, 2019). LLM is different from our ordinary software or API, which can objectively and effectively measure the performance of the model. Measuring the performance of LLM objectively and effectively presents a significant challenge. This encompasses evaluating the quality and correctness of the LLM's output content, as well as its alignment with human values. Finding objective indicators to quantify LLM performance remains a formidable task, making it a crucial challenge (Liu et al., 2023).

3.6. Alignment

Alignment refers to the process of training a model to generate text that is consistent with specific values or goals. Reinforcement learning from human feedback (RLHF) (Ouyang et al., 2022) is the main method to achieve alignment. Alignment is a training goal and is different from

fine-tuning in conception, which refers to the process of re-training LLMs on a small scale. RLHF is a category of fine-tuning. To achieve alignment between the model and human values, the main challenges include finding appropriate values, encoding values into AI systems, finding appropriate training data, etc.

Model alignment. It is imperative to ensure that the LLM adheres to human values, exhibiting expected behavior while avoiding adverse consequences. LLM may generate behaviors that are inconsistent with human values or intentions, which may lead to unexpected negative consequences. Therefore, the detection of misaligned behavior and methods to achieve model alignment are also a huge challenge (Bai et al., 2022; Bowman et al., 2022). LLM has superior performance, but we must also impose certain restrictions on it and cannot allow it to output content without constraints. The output of LLM must be compatible with the mainstream values of human beings. It should not output harmful content, or promote negative value orientations, especially for minors, even including some adults. Incorporating human values into the model training process is crucial to ensure that the AI system aligns with our ethical principles and moral standards. We need to achieve model alignment before the birth of autonomous artificial intelligence, because a poorly designed super artificial intelligence may quickly take control of the rational control and refuse any modification by its creator. As AI's learning and generalization capabilities are rapidly advancing, growing as fast as a rocket, there is a significant risk that it could cause substantial harm to human interests if it were to escape human control. AI may even find bugs in the system or loopholes in human rules to complete its training.

Safe alignment. To ensure the safe development of AI, we need targeted research in areas such as AI explainability, scalable supervision, and safety integration into the model-building process. This will help us create AI systems that are not only effective but also transparent, controllable, and safe. Ouyang et al. (2022). Perhaps we have seen in many films and television works, such as the *Terminator* and *Detroit: Become Human*, that the excessive development of AI has had a considerable impact on human society, such as AI becoming hostile to humans and destroying normal human life. During the model training process, we must also regard the safe development of the model as a necessary component. We cannot allow AI to grow wildly, out of human control, and cause immeasurable harm to humans. AI may also secretly evade the realization of goals when faced with multi-task goals after having its consciousness, leading to the failure of daily tasks. To achieve alignment between models and human values, the main challenges include finding appropriate values, encoding values into AI systems, obtaining appropriate training data, etc.

3.7. Performance prediction

The training time for LLMs has increased significantly in recent years due to their growing size and complexity. The training time of AI models has increased proportionally with the growing number of parameters and their scale. Training a model that meets the needs is a difficult process. Predicting model performance during LLM training is a challenging task that can lead to more efficient resource utilization. For example, we can build a demo model and use certain techniques to predict how will its performance change when scaled up. Maybe we can refer to ideas in software development, train a demo model, and then use certain technical methods to predict the performance of the model after it grows. If we can forecast the model's future performance during its early or mid-training stages, we can significantly reduce the amount of money, time, and resources spent on training. By predicting the model's eventual performance, we can avoid investing in models that are unlikely to meet our requirements, thus streamlining the training process and maximizing our returns. It can save more resources in the training process (Yang et al., 2024).

4. Conclusion

The rapid growth in computing power available in modern GPU clusters has made training trillion-parameter models a feasible reality, rather than a distant dream. In response, a 3D parallelism solution (Lai et al., 2023) has been proposed to tackle the training challenges of such massive models. This approach combines the strengths of three powerful techniques: data parallelism, model parallelism (tensor parallelism), and pipeline parallelism. By integrating these methods, 3D parallelism can simultaneously address the dual challenges of memory efficiency and computational efficiency when training trillion-parameter models. Building on the success of DeepSpeed (Rasley et al., 2020), which enables training of most LLMs without sacrificing speed, 3D parallelism leverages the complementary strengths of data, tensor, and pipeline parallelisms to optimize memory and compute efficiency. Specifically, each layer of the memory efficiency model is decomposed into pipeline stages, which are further divided into tensor parallelism units for efficient computation. Moreover, data parallelism can be expanded beyond model and pipeline parallelism, enabling further scalability.

As we move forward in training LLMs, we will likely place greater emphasis on mitigating the problem of overhead. Despite recent advances, the time and financial costs associated with training LLMs remain prohibitively high. However, if we can successfully reduce the costs associated with fine-tuning, it could have a significant impact on making these models more accessible to small and medium-sized enterprises, enabling them to leverage the benefits of LLMs without incurring exorbitant expenses. Their models are better able to be applied to downstream tasks. If we can first build a small demo, as is common in the field of software development, and then employ specific techniques to predict the model's performance after scaling up, we can save a certain amount of model training costs. Safety issues are also important considerations, particularly to ensure that LLM aligns with mainstream human values. There should be minimal discrimination or deviations leading to unexpected consequences, as otherwise, it may be unacceptable to humans.

CRediT authorship contribution statement

Fanlong Zeng: Writing – original draft, Investigation. **Wensheng Gan:** Writing – review & editing, Supervision. **Yongheng Wang:** Writing – original draft. **Philip S. Yu:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was supported in part by the National Natural Science Foundation of China (No. 62272196), and Guangzhou Basic and Applied Basic Research Foundation (No. 2024A04J9971).

References

Akyürek, E., Schuurmans, D., Andreas, J., Ma, T., Zhou, D., 2022. What learning algorithm is in-context learning? investigations with linear models. In: International Conference on Learning Representations.

Alajrami, A., Aletras, N., 2022. How does the pre-training objective affect what large language models learn about linguistic properties? In: Annual Meeting of the Association for Computational Linguistics.

Ananthaswamy, A., 2023. In AI, is bigger always better? Nature 615 (7951), 202–205.

Anwar, S., Hwang, K., Sung, W., 2017. Structured pruning of deep convolutional neural networks. ACM J. Emerg. Technol. Comput. Syst. 13 (3), 1–18.

Ardakani, A., Ji, Z., Smithson, S.C., Meyer, B.H., Gross, W.J., 2018. Learning recurrent binary/ternary weights. In: International Conference on Learning Representations.

Bahdanau, D., Cho, K., Bengio, Y., 2014. Neural machine translation by jointly learning to align and translate. In: The International Conference on Learning Representations.

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al., 2022. Constitutional AI: Harmlessness from AI feedback. arXiv preprint arXiv:2212.08073.

Beukeboom, C.J., Burgers, C., 2019. How stereotypes are shared through language: a review and introduction of the aocial categories and stereotypes communication (SCSC) framework. Rev. Commun. Res. 7, 1–37.

Boiko, D.A., MacKnight, R., Kline, B., Gomes, G., 2023. Autonomous chemical research with large language models. Nature 624 (7992), 570–578.

Borzunov, A., Baranchuk, D., Dettmers, T., Riabinin, M., Belkada, Y., Chumachenko, A., Samygin, P., Raffel, C., 2023. Petals: Collaborative inference and fine-tuning of large models. In: The Annual Meeting of the Association for Computational Linguistics. pp. 558–568.

Bowman, S.R., Hyun, J., Perez, E., Chen, E., Pettit, C., Heiner, S., Lukosuite, K., Askell, A., Jones, A., Chen, A., et al., 2022. Measuring progress on scalable oversight for large language models. arXiv preprint arXiv:2211.03540.

Brooks, T., Holynski, A., Efros, A.A., 2023. InstructPix2Pix: Learning to follow image editing instructions. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 18392–18402.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al., 2020. Language models are few-shot learners. Adv. Neural Inf. Process. Syst. 33, 1877–1901.

Buolamwini, J., Gebru, T., 2018. Gender shades: Intersectional accuracy disparities in commercial gender classification. In: The Conference on Fairness, Accountability and Transparency. PMLR, pp. 77–91.

Chen, Y., Gui, Y., Lin, H., Gan, W., Wu, Y., 2022. Federated learning attacks and defenses: A survey. In: IEEE International Conference on Big Data. IEEE, pp. 4256–4265.

Chen, Y., Huang, S., Gan, W., Huang, G., Wu, Y., 2023. Federated learning for metaverse: A survey. In: ACM Web Conference. pp. 1151–1160.

Chen, Q., Li, S., Gao, W., Sun, P., Wen, Y., Zhang, T., 2025. SPPO: Efficient long-sequence LLM training via adaptive sequence pipeline parallel offloading. arXiv preprint arXiv:2503.10377.

Chen, T., Li, M., Li, Y., Lin, M., Wang, N., Wang, M., Xiao, T., Xu, B., Zhang, C., Zhang, Z., 2015. MxNet: A flexible and efficient machine learning library for heterogeneous distributed systems. arXiv preprint arXiv:1512.01274.

Chen, T., Xu, B., Zhang, C., Guestrin, C., 2016. Training deep nets with sublinear memory cost. arXiv preprint arXiv:1604.06174.

Cheng, J., Gao, N., Yue, Y., Ye, Z., Jiang, J., Sha, J., 2024. EDiT: A local-SGD-based efficient distributed training method for large language models. arXiv preprint arXiv:2412.07210.

Cheng, Y., Wang, D., Zhou, P., Zhang, T., 2017. A survey of model compression and acceleration for deep neural networks. arXiv preprint arXiv:1710.09282.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H.W., Sutton, C., Gehrmann, S., et al., 2022. PaLM: Scaling language modeling with pathways. J. Mach. Learn. Res..

Chung, H.W., Hou, L., Longpre, S., Zoph, B., Tay, Y., Fedus, W., Li, E., Wang, X., Dehghani, M., Brahma, S., et al., 2022. Scaling instruction-finetuned language models. J. Mach. Learn. Res. 25, 70:1–70:53.

Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L., 2023. QLoRA: Efficient finetuning of quantized LLMs. Adv. Neural Inf. Process. Syst. 36, 10088–10115.

Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2018. BERT: Pre-training of deep bidirectional transformers for language understanding. In: The North American Chapter of the Association for Computational Linguistics: Human Language Technologies.

Dodge, J., Prewitt, T., Tachet des Combes, R., Odmark, E., Schwartz, R., Strubell, E., Luccioni, A.S., Smith, N.A., DeCario, N., Buchanan, W., 2022. Measuring the carbon intensity of AI in cloud instances. In: ACM Conference on Fairness, Accountability, and Transparency. pp. 1877–1894.

Dong, Y., Ma, J., Wang, S., Chen, C., Li, J., 2023. Fairness in graph mining: A survey. IEEE Trans. Knowl. Data Eng. 35 (10), 10583–10602.

Dong, Z., Yao, Z., Gholami, A., Mahoney, M.W., Keutzer, K., 2019. HAWQ: Hessian aware quantization of neural networks with mixed-precision. In: IEEE/CVF International Conference on Computer Vision. pp. 293–302.

Dozat, T., 2016. Incorporating nesterov momentum into ADMA.

Du, M., He, F., Zou, N., Tao, D., Hu, X., 2022. Shortcut learning of large language models in natural language understanding: A survey. Commun. ACM.

Duchi, J., Hazan, E., Singer, Y., 2011. Adaptive subgradient methods for online learning and stochastic optimization. J. Mach. Learn. Res. 12 (7).

Fang, G., Ma, X., Song, M., Mi, M.B., Wang, X., 2023. DepGraph: Towards any structural pruning. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 16091–16101.

Frantar, E., Alistarh, D., 2023. Massive language models can be accurately pruned in one-shot. In: International Conference on Machine Learning.

Gan, W., Chen, L., Wan, S., Chen, J., Chen, C.-M., 2023a. Anomaly rule detection in sequence data. IEEE Trans. Knowl. Data Eng. 35 (12), 12095–12108.

Gan, W., Huang, G., Weng, J., Gu, T., Yu, P.S., 2025a. Towards target sequential rules. IEEE Trans. Knowl. Data Eng. 37 (6), 3766–3780.

Gan, W., Ning, Z., Qi, Z., Yu, P.S., 2025b. Mixture of experts (MoE): A big data perspective. arXiv preprint arXiv:2501.16352.

Gan, W., Qi, Z., Wu, J., Lin, J.C.-W., 2023b. Large language models in education: Vision and opportunities. In: IEEE International Conference on Big Data. IEEE, pp. 4776–4785.

Gan, W., Wan, S., Yu, P., 2023c. Model-as-a-service (MaaS): A survey. In: IEEE International Conference on Big Data. IEEE, pp. 4636–4645.

Garfinkel, S., Abowd, J.M., Martindale, C., 2018. Understanding database reconstruction attacks on public data: These attacks on statistical databases are no longer a theoretical danger. Queue 16 (5), 28–53.

Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., Wichmann, F.A., 2020. Shortcut learning in deep neural networks. Nat. Mach. Intell. 2 (11), 665–673.

Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M.W., Keutzer, K., 2022. A survey of quantization methods for efficient neural network inference. In: Low-Power Computer Vision. Chapman and Hall/CRC, pp. 291–326.

Gibiansky, A., 2017. Bringing HPC techniques to deep learning. Tech. Rep., Baidu Research.

Godoy, W.F., Valero-Lara, P., Teranishi, K., Balaprakash, P., Vetter, J.S., 2023. Evaluation of OpenAI Codex for HPC parallel programming models kernel generation. In: International Conference on Parallel Processing Workshops.

Gugger, S., Debut, L., Wolf, T., Schmid, P., Mueller, Z., Mangrulkar, S., Sun, M., Bossan, B., 2022. Accelerate: Training and inference at scale made simple, efficient and adaptable. https://github.com/huggingface/accelerate.

Gumaan, E., 2025. Galvatron: Automatic distributed training for large transformer models. arXiv preprint arXiv:2504.03662.

Hu, X., Chen, J., Li, X., Guo, Y., Wen, L., Yu, P.S., Guo, Z., 2023. Do large language models know about facts?. arXiv preprint arXiv:2310.05177.

Hu, H., Salcic, Z., Sun, L., Dobbie, G., Yu, P.S., Zhang, X., 2022. Membership inference attacks on machine learning: A survey. ACM Comput. Surv. 54 (11s), 1–37.

Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, D., Chen, M., Lee, H., Ngiam, J., Le, Q.V., Wu, Y., et al., 2019. GPipe: Efficient training of giant neural networks using pipeline parallelism. Adv. Neural Inf. Process. Syst. 32.

Huang, W., Wang, Y., Cheng, A., Zhou, A., Yu, C., Wang, L., 2024. A fast, performant, secure distributed training framework for large language model. arXiv preprint arXiv:2401.09796.

Iacob, A., Sani, L., Marino, B., Aleksandrov, P., Shen, W.F., Lane, N.D., 2024. Worldwide Federated Training of Language Models. In: International Workshop on Federated Foundation Models in Conjunction with NeurIPS 2024.

Idelbayev, Y., Carreira-Perpinán, M.A., 2020. Low-rank compression of neural nets: Learning the rank of each layer. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 8049–8059.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., Amodei, D., 2020. Scaling laws for neural language models. arXiv preprint arXiv:2001.08361.

Khirirat, S., Feyzmahdavian, H.R., Johansson, M., 2017. Mini-batch gradient descent: Faster convergence under data sparsity. In: IEEE Annual Conference on Decision and Control. IEEE, pp. 2880–2887.

Kim, M., Lee, S., Hong, S., Chang, D.-S., Choi, J., 2022. Understanding and improving knowledge distillation for quantization-aware training of large transformer encoders. In: Conference on Empirical Methods in Natural Language Processing.

Kingma, D.P., Ba, J., 2014. ADAM: A method for stochastic optimization. In: The International Conference on Learning Representations.

Korthikanti, V.A., Casper, J., Lym, S., McAfee, L., Andersch, M., Shoeybi, M., Catanzaro, B., 2023. Reducing activation recomputation in large transformer models. Proc. Mach. Learn. Syst. 5.

Kotipalli, P.V., Singh, R., Wood, P., Laguna, I., Bagchi, S., 2019. AMPT-GA: automatic mixed precision floating point tuning for GPU applications. In: ACM International Conference on Supercomputing. pp. 160–170.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I., 2023. Efficient memory management for large language model serving with PagedAttention. In: ACM SIGOPS Symposium on Operating Systems Principles.

Lai, J., Gan, W., Wu, J., Qi, Z., Yu, P.S., 2024. Large language models in law: A survey. AI Open 5, 181–196.

Lai, Z., Li, S., Tang, X., Ge, K., Liu, W., Duan, Y., Qiao, L., Li, D., 2023. Merak: An efficient distributed dnn training framework with automated 3d parallelism for giant foundation models. IEEE Trans. Parallel Distrib. Syst. 34 (5), 1466–1478.

LeCun, Y., Bengio, Y., Hinton, G., 2015. Deep learning. Nature 521 (7553), 436–444.

Lee, H., Li, S., Vu, N.T., 2022. Meta learning for natural language processing: A survey. In: Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies.

Li, C., 2020a. OpenAI's GPT-3 Language Model: A Technical Overview. Blog Post.

Li, C., 2020b. OpenAI's GPT-3 language model: A technical overview. https://lambdalabs.com/blog/demystifying-gpt-3.

Li, M., Andersen, D.G., Park, J.W., Smola, A.J., Ahmed, A., Josifovski, V., Long, J., Shekita, E.J., Su, B.-Y., 2014. Scaling distributed machine learning with the parameter server. In: USENIX Symposium on Operating Systems Design and Implementation. pp. 583–598.

Li, H., Chen, L., Ma, D., Wu, Z., Zhu, S., Yu, K., 2024. Evolving subnetwork training for large language models. In: The International Conference on Machine Learning. pp. 27547–27562.

Li, H., Kadav, A., Durdanovic, I., Samet, H., Graf, H.P., 2016. Pruning filters for efficient convnets. In: International Conference on Learning Representations.

Li, S., Liu, H., Bian, Z., Fang, J., Huang, H., Liu, Y., Wang, B., You, Y., 2023. Colossal-AI: A unified deep learning system for large-scale parallel training. In: International Conference on Parallel Processing. pp. 766–775.

Lian, X., Zhang, W., Zhang, C., Liu, J., 2018. Asynchronous decentralized parallel stochastic gradient descent. In: International Conference on Machine Learning. PMLR, pp. 3043–3052.

Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., et al., 2023. Holistic evaluation of language models. Trans. Mach. Learn. Res. ppt. 1–xx.

Liu, J., Liu, C., Zhou, P., Ye, Q., Chong, D., Zhou, K., Xie, Y., Cao, Y., Wang, S., You, C., et al., 2023. LLMRec: Benchmarking large language models on recommendation task. arXiv preprint arXiv:2308.12241.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V., 2019. RoBERTa: A robustly optimized BERT pretraining approach. arXiv preprint arXiv:1907.11692.

Liu, Z., Wang, Y., Han, K., Zhang, W., Ma, S., Gao, W., 2021. Post-training quantization for vision transformer. Adv. Neural Inf. Process. Syst. 34, 28092–28103.

Lu, J., Yang, J., Batra, D., Parikh, D., 2016. Hierarchical question-image co-attention for visual question answering. Adv. Neural Inf. Process. Syst. 29.

Luo, Q., He, J., Zhuo, Y., Qian, X., 2020. Prague: High-performance heterogeneity-aware asynchronous decentralized training. In: International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 401–416.

Luong, M.-T., Pham, H., Manning, C.D., 2015. Effective approaches to attention-based neural machine translation. In: The Conference on Empirical Methods in Natural Language Processing.

Ma, X., Fang, G., Wang, X., 2023. LLM-Pruner: On the structural pruning of large language models. Adv. Neural Inf. Process. Syst.

McCallum, S., 2022. ChatGPT banned in Italy over privacy concerns. https://apnews.com/article/chatgpt-ai-data-privacy-italy-66634e4d9ade3c0eb63edab62915066f.

MindSpeed-LLM, 2023. MindSpeed-LLM. [Online]. Available: https://gitee.com/ascend/MindSpeed-LLM. https://gitee.com/ascend/MindSpeed-LLM.

Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M.I., et al., 2018. Ray: A distributed framework for emerging AI applications. In: USENIX Symposium on Operating Systems Design and Implementation. pp. 561–577.

Narang, S., Diamos, G., Elsen, E., Micikevicius, P., Alben, J., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., et al., 2018. Mixed precision training. In: The International Conference on Learning Representations.

Narayanan, D., Harlap, A., Phanishayee, A., Seshadri, V., Devanur, N.R., Ganger, G.R., Gibbons, P.B., Zaharia, M., 2019. PipeDream: Generalized pipeline parallelism for DNN training. In: ACM Symposium on Operating Systems Principles. pp. 1–15.

Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., et al., 2021. Efficient large-scale language model training on GPU clusters using megatron-lm. In: International Conference for High Performance Computing, Networking, Storage and Analysis. pp. 1–15.

NVIDIA, 2023. Megatron-DeepSpeed, https://github.com/microsoft/Megatron-DeepSpeed#megatron-deepspeed.

openAI, 2023. Production best practice-improve latency. https://platform.openai.com/docs/guides/production-best-practices/improving-latencies.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al., 2022. Training language models to follow instructions with human feedback. Adv. Neural Inf. Process. Syst. 35, 27730–27744.

Pungas, T., 2023. GPT-3.5 and GPT-4 response times. https://www.taivo.ai/_gpt-3-5-and-gpt-4-response-imes/.

Qian, N., 1999. On the momentum term in gradient descent learning algorithms. Neural Netw. 12 (1), 145–151.

Qiu, S., Liu, Q., Zhou, S., Huang, W., 2022. Adversarial attack and defense technologies in natural language processing: A survey. Neurocomputing 492, 278–307.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J., 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. J. Mach. Learn. Res. 21 (1), 5485–5551.

Rajbhandari, S., Li, C., Yao, Z., Zhang, M., Aminabadi, R.Y., Awan, A.A., Rasley, J., He, Y., 2022. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In: The International Conference on Machine Learning. PMLR, pp. 18332–18346.

Rajbhandari, S., Rasley, J., Ruwase, O., He, Y., 2020. ZeRO: Memory optimizations toward training trillion parameter models. In: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, pp. 1–16.

Rajbhandari, S., Ruwase, O., Rasley, J., Smith, S., He, Y., 2021. Zero-Infinity: Breaking the GPU memory wall for extreme scale deep learning. In: International Conference for High Performance Computing, Networking, Storage and Analysis. pp. 1–14.

Rasley, J., Rajbhandari, S., Ruwase, O., He, Y., 2020. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In: The ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 3505–3506.

Ren, J., Rajbhandari, S., Aminabadi, R.Y., Ruwase, O., Yang, S., Zhang, M., Li, D., He, Y., 2021. ZeRO-offload: Democratizing billion-scale model training. In: USENIX Annual Technical Conference. pp. 551–564.

Ruder, S., 2016. An overview of gradient descent optimization algorithms. arXiv preprint arXiv:1609.04747.

Ruder, S., Peters, M.E., Swayamdipta, S., Wolf, T., 2019. Transfer learning in natural language processing. In: Conference of the North American Chapter of the Association for Computational Linguistics: Tutorials. pp. 15–18.

Sani, L., Iacob, A., Cao, Z., Marino, B., Gao, Y., Paulik, T., Zhao, W., Shen, W.F., Aleksandrov, P., Qiu, X., et al., The Future of Large Language Model Pre-training is Federated, In: International Workshop on Federated Foundation Models in Conjunction with NeurIPS.

Scao, T.L., Fan, A., Akiki, C., Pavlick, E., Ilić, S., Hesslow, D., Castagné, R., Luccioni, A.S., Yvon, F., Gallé, M., et al., 2022. BLOOM: A 176B-parameter open-access multilingual language model. arXiv preprint arXiv:2211.05100.

Schaeffer, R., Miranda, B., Koyejo, S., 2023. Are emergent abilities of large language models a mirage? Adv. Neural Inform. Process. Syst.

Schatz, M.D., Poulson, J., van de Geijn, R.A., 2012. Scalable universal matrix multiplication algorithms: 2D and 3D variations on a theme. ACM Trans. Math. Software 1–30.

Schaul, T., Zhang, S., LeCun, Y., 2013. No more pesky learning rates. In: International Conference on Machine Learning. PMLR, pp. 343–351.

Sergeev, A., Del Balso, M., 2018. Horovod: fast and easy distributed deep learning in TensorFlow. arXiv preprint arXiv:1802.05799.

Shen, Y., Song, K., Tan, X., Li, D., Lu, W., Zhuang, Y., 2023. HugginGPT: Solving ai tasks with chatgpt and its friends in hugging face. Adv. Neural Inf. Process. Syst. 36, 38154–38180.

Song, J., Yim, J., Jung, J., Jang, H., Kim, H.-J., Kim, Y., Lee, J., 2023. Optimus-CC: Efficient large nlp model training with 3d parallelism aware communication compression. In: International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 2, pp. 560–573.

Sun, W., Qin, Z., Sun, W., Li, S., Li, D., Shen, X., Qiao, Y., Zhong, Y., 2024. CO2: Efficient distributed training with full communication-computation overlap. In: International Conference on Learning Representations.

Tang, R., Chuang, Y.-N., Hu, X., 2024. The science of detecting LLM-generated text. Commun. ACM 67 (4), 50–59.

Tang, R., Du, M., Li, Y., Liu, Z., Zou, N., Hu, X., 2021. Mitigating gender bias in captioning systems. In: ACM Web Conference. pp. 633–645.

Tieleman, T., Hinton, G., 2017. Divide the Gradient by a Running Average of Its Recent Magnitude. Technical Report.

Truong, J.-B., Maini, P., Walls, R.J., Papernot, N., 2021. Data-free model extraction. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 4771–4780.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. Adv. Neural Inf. Process. Syst. 30.

Wagenländer, M., Li, G., Zhao, B., Mai, L., Pietzuch, P., 2024. TENPLEX: Dynamic parallelism for deep learning using parallelizable tensor collections. In: Symposium on Operating Systems Principles. pp. 195–210.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., Zhou, D., 2022a. Self-consistency improves chain of thought reasoning in language models. In: International Conference on Learning Representations.

Wang, X., Xiong, Y., Wei, Y., Wang, M., Li, L., 2021. LightSeq: A high performance inference library for transformers. In: Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Industry Papers. pp. 113–120.

Wang, B., Xu, Q., Bian, Z., You, Y., 2022b. Tesseract: Parallelize the tensor parallelism efficiently. In: International Conference on Parallel Processing. pp. 1–11.

Wang, H., Zhou, J., Liu, W., Li, J., Huang, X., Liu, L., Liang, W., Yu, C., Li, F., Li, Z., 2020. BGD-based Adam algorithm for time-domain equalizer in PAM-based optical interconnects. Opt. Lett. 45 (1), 141–144.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., et al., 2022. Chain-of-thought prompting elicits reasoning in large language models. Adv. Neural Inf. Process. Syst. 35, 24824–24837.

Winston, P.H., 1984. Artificial Intelligence. Addison-Wesley Longman Publishing Co. Inc.

Wu, J., Gan, W., Chen, Z., Wan, S., Yu, P.S., 2023. Multimodal large language models: A survey. In: IEEE International Conference on Big Data. IEEE, pp. 2247–2256.

Wu, B., Wang, S., Tang, Y., Ding, J., Helenowski, E., Tan, L., Xu, T., Gowda, T., Chen, Z., Zhu, C., et al., 2025. LlamaRL: A distributed asynchronous reinforcement learning framework for efficient large-scale LLM trainin. arXiv preprint arXiv:2505.24034.

Xiong, W., Lagerström, R., 2019. Threat modeling–a systematic literature review. Comput. Secur. 84, 53–69.

Xu, K., Ba, J., Kiros, R., Cho, K., Courville, A., Salakhutdinov, R., Zemel, R., Bengio, Y., 2015. Neural image caption generation with visual attention. In: International Conference on Learning Representations. pp. 2048–2057.

Xu, H., Gan, W., Qi, Z., Wu, J., Yu, P.S., 2024. Large language models for education: A survey. arXiv preprint arXiv:2405.13001.

Xu, Q., You, Y., 2023. An efficient 2D method for training super-large deep learning models. In: IEEE International Parallel and Distributed Processing Symposium. IEEE, pp. 222–232.

Yang, J., Jin, H., Tang, R., Han, X., Feng, Q., Jiang, H., Zhong, S., Yin, B., Hu, X., 2024. Harnessing the power of LLMs in practice: A survey on ChatGPT and beyond. ACM Trans. Knowl. Discov. Data 18 (6), 1–32.

Yang, Z., Yang, D., Dyer, C., He, X., Smola, A., Hovy, E., 2016. Hierarchical attention networks for document classification. In: Conference of the North American Chapter of the Association for Computational Linguistics. pp. 1480–1489.

Ye, R., Wang, W., Chai, J., Li, D., Li, Z., Xu, Y., Du, Y., Wang, Y., Chen, S., 2024. OpenFedLLM: Training large language models on decentralized private data via federated learning. In: The Conference on Knowledge Discovery and Data Mining. pp. 6137–6147.

Yin, W., Hay, J., Roth, D., 2019. Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach. In: Conference on Empirical Methods in Natural Language Processing and the International Joint Conference on Natural Language Processing.

Zeiler, M.D., 2012. ADADELTA: an adaptive learning rate method. arXiv preprint arXiv:1212.5701.

Zeng, F., Gan, W., Wang, Y., Liu, N., Yu, P.S., 2023a. Large language models for robotics: A survey. arXiv preprint arXiv:2311.07226.

Zeng, F., Gan, W., Wang, Y., Yu, P.S., 2023b. Distributed training of large language models. In: IEEE 29th International Conference on Parallel and Distributed Systems. IEEE, pp. 840–847.

Zha, D., Bhat, Z.P., Lai, K.-H., Yang, F., Jiang, Z., Zhong, S., Hu, X., 2025. Data-centric artificial intelligence: A survey. ACM Comput. Surv. 57 (5), 1–42.

Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X.V., et al., 2022. OPT: Open pre-trained transformer language models. arXiv preprint arXiv:2205.01068.

Zhang, M., Shen, C., Yang, Z., Ou, L., Yu, X., Zhuang, B., et al., 2023. Pruning meets low-rank parameter-efficient fine-tuning. arXiv preprint arXiv:2305.18403.

Zhang, T., Ye, S., Zhang, K., Tang, J., Wen, W., Fardad, M., Wang, Y., 2018. A systematic DNN weight pruning framework using alternating direction method of multipliers. In: European Conference on Computer Vision. pp. 184–199.

Zhao, B.Z.H., Agrawal, A., Coburn, C., Asghar, H.J., Bhaskar, R., Kaafar, M.A., Webb, D., Dickinson, P., 2021. On the (in) feasibility of attribute inference attacks on machine learning models. In: IEEE European Symposium on Security and Privacy. IEEE, pp. 232–251.

Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C.-C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., et al., 2023. Pytorch FSDP: experiences on scaling fully sharded data parallel. VLDB Endow..

Zheng, Y., Gan, W., Chen, Z., Qi, Z., Liang, Q., Yu, P.S., 2025. Large language models for medicine: a survey. Int. J. Mach. Learn. Cybern. 16 (2), 1015–1040.

Zhu, Z., Giannoula, C., Andoorveedu, M., Su, Q., Mangalam, K., Zheng, B., Pekhimenko, G., 2025. Mist: Efficient distributed training of large language models via memory-parallelism co-optimization. In: European Conference on Computer Systems.

Zhu, X., Li, J., Liu, Y., Ma, C., Wang, W., 2024a. A Survey on model compression for arge language models. Trans. Assoc. Comput. Linguist. 12, 1556–1577.

Zhu, Y., Wu, L., Guo, Q., Hong, L., Li, J., 2024b. Collaborative large language model for recommender systems. In: ACM on Web Conference. pp. 3162–3172.